

HRNN: A Hybrid Graph Index for Efficient Reverse k -Nearest Neighbor Search on High-Dimensional Vectors

Wenxuan Xia

HKUST (GZ)

China

wxia248@connect.hkust-gz.edu.cn

Wentao Li

University of Leicester

United Kingdom

wl226@leicester.ac.uk

Mingyu Yang

HKUST (GZ) & HKUST

China

myang250@connect.hkust-gz.edu.cn

Wei Wang

HKUST (GZ) & HKUST

China

weiwcs@ust.hk

ABSTRACT

Reverse k -nearest neighbor (RkNN) search returns all data vectors that regard a query vector as one of their k -nearest neighbors (k NNs). Existing RkNN methods typically follow a **filter-and-verification framework**: data vectors near the query vector are first collected as candidates and then verified against their k NN-radius (i.e., the distance to their k -th nearest neighbor). However, existing methods face two key limitations in high-dimensional spaces. First, nearby vectors often do not belong to the query’s true RkNN set, resulting in excessive candidate expansion overhead. Second, existing methods compute k NN-radius online during verification, incurring substantial query-processing cost.

To address these limitations, we propose HRNN, a hybrid graph index for approximate RkNN search. (1) Rather than directly treating nearby vectors as RkNN candidates, HRNN uses them as proxy vectors based on the **assumption** that a query’s RkNN results can often be discovered through the RkNN results of its nearby vectors. (2) To reduce verification cost, HRNN **materializes** high-fidelity k NN-radius offline, eliminating expensive online reconstruction while preserving accuracy. HRNN combines a navigation graph, a ranked KNN graph, and reverse-neighbor lists into a hybrid index that supports efficient proxy retrieval, candidate generation, and k NN-radius access. We also develop efficient algorithms for index construction and maintenance. Extensive experiments show that HRNN consistently outperforms existing methods, achieving up to one order of magnitude higher throughput. Moreover, HRNN scales to massive datasets containing millions of high-dimensional vectors while supporting efficient dynamic index maintenance.

PVLDB Reference Format:

Wenxuan Xia, Mingyu Yang, Wentao Li, and Wei Wang. HRNN: A Hybrid Graph Index for Efficient Reverse k -Nearest Neighbor Search on High-Dimensional Vectors. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/gegeji/hrnn>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

With the rapid development of large language models (LLMs) and intelligent agents [32, 56], modern data are increasingly represented as high-dimensional vectors, such as text embeddings, image features, and other multi-modal representations [3, 72, 74]. Given a dataset D of data vectors/points, the **k -nearest neighbors (k NN)** of a data point are the k data points in D closest to it, where distance is typically measured using Euclidean distance. Conversely, the **reverse k -nearest neighbors (RkNN)** of a query vector q consist of all data points in D that regard q as one of their k -nearest neighbors. Given a dataset D , the RkNN search problem aims to efficiently retrieve the RkNN set of a query vector q from D .

The RkNN set of a query vector q naturally reflects its influence over the dataset D , since each vector in the RkNN set regards q as one of its k -nearest neighbors. Consequently, RkNN relationships serve as a computational primitive in machine learning and high-dimensional data analysis. (1) In machine learning, reverse neighbors guide representative sample selection for graph neural network explanation [2], graph condensation [20], and lane-image annotation [45]. They are also used to learn discriminative feature projections [22, 77] and fine-tune neural networks under label noise [31, 41]. (2) In high-dimensional data analysis, hubness [37, 47, 54] has been exploited in downstream tasks such as multi-view clustering [73] and GAN latent sampling [37]. Hubness characterizes the tendency of certain points, known as hubs, to occur frequently among the k -nearest neighbors of other points. Since hubs correspond to points with large RkNN sets, the RkNN search provides a direct means of identifying them.

To process the RkNN search for a query q , an **intuitive** approach is to first compute the k NN set of every point o in the dataset D and obtain its **k NN-radius**, i.e., the distance between o and its k -th nearest neighbor. Then, by definition, if the distance between q and point $o \in D$ does not exceed the k NN-radius of o , o is identified as an RkNN of q . Yet, this intuitive approach becomes prohibitively expensive when the dataset is large or the vector dimensionality is high. Although prior studies [7, 49] have explored spatial indexes such as R-trees to accelerate RkNN search, exact RkNN retrieval in high-dimensional vector spaces remains challenging. Thus, this paper focuses on approximate RkNN (i.e., ARkNN) search.

Existing Solutions. Existing methods for ARkNN search generally follow a **filter-and-verification framework**. They first retrieve a

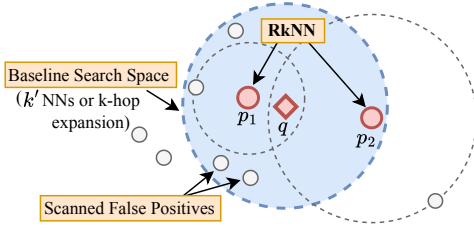


Figure 1: Illustration of the inefficiency of existing methods. The circles centered at data points (e.g., p_1 and p_2) represent their k NN-radius. Given a query vector q , the goal is to retrieve its $RkNN$ s (p_1 and p_2 for $k = 1$). Existing methods typically explore nearby data points via top- k' ($k' \gg k$) NN search or k -hop graph expansion, followed by candidate verification. However, reaching p_2 , whose sparse neighborhood yields a large k NN-radius, requires exploring a large region and examining many false positives (gray points).

superset of potential $RkNN$ results for a query q as candidates, and then verify each candidate using a process similar to the intuitive approach described above. Early methods mainly rely on spatial partitioning [51, 52] or precomputed distance bounds [1] to prune unpromising points from the dataset. Yet, these approaches suffer from high preprocessing and indexing costs in high-dimensional spaces due to the curse of dimensionality [24]. More recent methods, including SFT [48], RDT [6], and the state-of-the-art HAMG [50], are based on the assumption that **the $RkNN$ s of a query vector often overlap significantly with its nearby neighbors**. Thus, these methods first retrieve nearby neighbors as candidates, such as the top- k' nearest neighbors of the query vector in SFT [48] and RDT [6], where $k' \gg k$, or the k -hop neighbors in graph-based methods such as HAMG [50]. They then verify whether each candidate belongs to the $RkNN$ set of the query by comparing the query distance against the candidate’s k NN-radius.

Limitations. Although recent methods achieve good performance by considering the properties of high-dimensional spaces, they still suffer from inherent limitations that restrict their efficiency.

Limitation 1: The mismatch between $RkNN$ and nearby neighbors.

Existing methods need to examine a large number of nearby neighbors to achieve high recall, because true $RkNN$ results can be far away from the query vector in the embedding space (see Fig. 1 and Fig. 4). For example, a point $o \in D$ may still regard query vector q as one of its k -nearest neighbors if o lies in a sparse region and therefore has a large k NN-radius. As a result, the query must search a large neighborhood region before reaching o , leading to excessive candidate expansion and unnecessary verification cost. This mismatch between $RkNN$ results and nearby neighbors fundamentally limits the efficiency of existing methods.

Limitation 2: Costly online k NN-radius computation. After generating candidate points, existing methods verify whether a candidate o belongs to the final $RkNN$ result by comparing the distance between query vector q and candidate point o against the k NN-radius of o . Specifically, if $\delta(q, o)$ is no larger than the k NN-radius of o , then q is regarded as one of the k -nearest neighbors of o , and thus o is an $RkNN$ of query q . Yet, the k NN-radius of each candidate is typically computed online during query time [6, 50, 68], which incurs substantial overhead when the candidate set is large, as the k NN-radius must be computed online for each candidate.

Table 1: Comparison between HRNN and existing $RkNN$ methods.

	HRNN (Ours)	HAMG	RDT	SFT
Filter (Candidate Generation)	Reverse-neighbor lists of proxies	k -hop graph traversal	Incremental k NN search	One-shot k NN search
Verification	Pre-materialized k NN-radius	Online computation	Online computation	Online computation

Our Solution. To address the limitations of existing methods, we propose HRNN, a Hybrid graph index for approximate Reverse k -Nearest Neighbor search. (1) To overcome the mismatch between $RkNN$ results and nearby neighbors, rather than assuming that a query’s nearby neighbors directly contain its $RkNN$ results, we make a different assumption: **a query’s $RkNN$ results can often be discovered through the $RkNN$ results of its nearby neighbors**. Based on this assumption, HRNN first retrieves the query’s nearby neighbors (i.e., k NNs) as **proxy** points and then leverages their $RkNN$ results to recover the query’s $RkNN$ results. To support this process, each point o maintains a **reverse-neighbor list** containing the vectors in D whose K -nearest-neighbor lists include o , enabling efficient retrieval of its $RkNN$ results for any $k \leq K$ when used as a proxy, where K is a build-time hyperparameter of HRNN. This design avoids excessive neighborhood expansion while maintaining high retrieval accuracy. (2) The k NN-radius is required to verify if each candidate is a true $RkNN$ result. To avoid costly online k NN-radius computation, HRNN materializes this information offline using the precomputed nearest-neighbor relationships.

Achieving these goals is non-trivial, as the index must simultaneously support k NN retrieval for any query vectors, reverse-neighbor-list access for any proxy points, and efficient k NN-radius lookup for any $k \leq K$. To this end, HRNN integrates two complementary graph structures. First, it employs a navigation graph, such as HNSW [42], to retrieve the query vector’s nearest neighbors as proxy points. Second, it maintains a ranked KNN graph [13] that stores nearest-neighbor relationships among data vectors in D . This graph provides direct access to pre-materialized k NN-radius for verification and serves as the basis for constructing reverse-neighbor lists, which enable efficient retrieval of proxy points’ $RkNN$ results for candidate generation. By integrating a navigation graph, a ranked KNN graph, and reverse-neighbor lists into a unified index, HRNN enables efficient $RkNN$ query processing. We further develop efficient algorithms for index construction and maintenance under data point insertions and deletions. A comparison between HRNN and existing methods is in Table 1.

Contributions. We summarize our main contributions as follows.

Problem analysis (§3): We identify two efficiency bottlenecks of existing $ARkNN$ methods: the mismatch between $RkNN$ results and nearby neighbors for the query vector, and the costly online computation of k NN-radius during the candidate verification.

Hybrid index design (§4): To address these limitations, we propose HRNN, a novel index for $RkNN$ search. HRNN integrates a navigation graph, a ranked KNN graph, and reverse-neighbor lists to support proxy retrieval, candidate generation, and efficient verification using materialized k NN-radius. We further show how HRNN supports efficient $RkNN$ search.

Table 2: Notation summary.

Notation	Description
G_{HNSW}	HNSW navigation graph
G_{KNN}	Ranked KNN graph
$G_{\text{KNN}}[o]$	Ranked KNN list of o , sorted by distance
$R[o]$	Reverse-neighbor list of o
$r_k(o)$	k NN-radius (distance to the k -th nearest neighbor) of o
k	Target parameter for k NN or Rk NN search
K	Number of stored neighbors per point in G_{KNN}
Θ	Rank threshold for reverse candidate generation
m	Number of proxy points retrieved from G_{HNSW}
Θ_u	Rank threshold used during index maintenance
m_u	Number of update proxies used during index maintenance

Index construction and maintenance (§4): We develop efficient algorithms for index construction, as well as for index maintenance under data point insertions and deletions.

Extensive experiments (§5): Experiments on eight real-world datasets show that HRNN improves the accuracy–throughput trade-off over strong baselines by up to an order of magnitude and scales to datasets containing millions of vectors.

2 PRELIMINARY

We first formulate the approximate Rk NN search problem, and then introduce the graph-based index that supports k NN retrieval.

2.1 Problem Definition

We first introduce the definitions of k NN and Rk NN, and then formally define the studied problem. Table 2 summarizes the commonly used notations in this paper.

Definition 2.1 (k -Nearest Neighbor (k NN)). Given a dataset D of vectors/points in Euclidean space $\mathbb{R}^d$, a d -dimensional query vector q , and an integer k , the k NN set $N_k(q)$ of q is a subset of D such that $|N_k(q)| = k$ and $\forall v \in N_k(q), \forall x \in D \setminus N_k(q), \delta(v, q) \leq \delta(x, q)$, where $\delta(v, q)$ denotes the Euclidean distance between v and q .

Note that q may not belong to D . For each point $o \in D$, let $N_k(o)$ denote its k NN in D , and let its **k NN-radius** be defined as $r_k(o) = \delta(o, v)$, where $v \in N_k(o)$ is the k -th nearest neighbor of o .

Definition 2.2 (Reverse k -Nearest Neighbor (Rk NN)). Given a dataset $D \subseteq \mathbb{R}^d$, a d -dimensional query vector q , and an integer k , the Rk NN set of q is defined as $A_k(q) = \{o \in D \mid \delta(q, o) \leq r_k(o)\}$.

The Rk NN set $A_k(q)$ contains all points $o \in D$ that regard q as one of their k nearest neighbors. Equivalently, the distance between q and o is no larger than the k NN-radius of point o . Since exact Rk NN search is computationally expensive in high-dimensional spaces, existing methods typically trade accuracy for efficiency by returning approximate results. We therefore formally define the problem of **approximate Rk NN (ARkNN) search** as follows.

Definition 2.3 (Approximate Reverse k -Nearest Neighbor (ARkNN) Search). Given a dataset D , a query vector q , and an integer k , an ARkNN search returns a set $\hat{A}_k(q) \subseteq D$ as an approximation to the exact Rk NN result set $A_k(q)$.

To evaluate the accuracy of the returned ARkNN results, we use Recall@ k and Precision@ k against the exact Rk NN ground truth.

Definition 2.4. For a query vector q with exact Rk NN result set $A_k(q)$ and approximate result set $\hat{A}_k(q)$, we define $\text{Recall}@k(q) = \frac{|A_k(q) \cap \hat{A}_k(q)|}{|\hat{A}_k(q)|}$, $\text{Precision}@k(q) = \frac{|A_k(q) \cap \hat{A}_k(q)|}{|A_k(q)|}$.

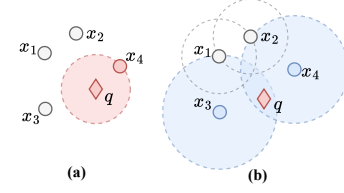


Figure 2: Illustrative examples of k NN and Rk NN search.

Recall@ k measures the fraction of exact Rk NN results successfully retrieved, whereas Precision@ k measures the fraction of retrieved results that are exact Rk NNs. If the denominator set is empty, the corresponding metric is defined as 1 when both sets are empty and 0 otherwise. We report both metrics averaged over all queries.

Example 2.5. Fig. 2 shows k NN and Rk NN search. Let $x_1, \dots, x_4$ be data points and q the query, with $k = 1$. In Fig. 2(a), the nearest neighbor of q is x_4 , and the (radius of the) circle centered at q denotes its k NN-radius. In Fig. 2(b), the circles centered at $x_1, \dots, x_4$ indicate their k NN-radius. Since q lies within the k NN-radius of x_3 and x_4 but outside those of x_1 and x_2 , the Rk NN result set of q is $\{x_3, x_4\}$.

2.2 Graph-Based k NN Search

We briefly introduce graph-based methods for high-dimensional k NN search, which serve as the foundation of the studied ARkNN search problem. Among existing high-dimensional k NN methods, graph-based approaches [8, 13, 15, 16, 19, 21, 25, 26, 33, 38, 40, 42, 46, 57, 60, 61, 76] are particularly popular due to their state-of-the-art performance. These methods model the points in dataset D as vertices in a graph and connect nearby points through directed or undirected edges. As a result, k NN search for a query vector q becomes a graph traversal process over the constructed index.

Ranked KNN Graph. We first introduce the ranked KNN graph, where K is a user-defined parameter chosen to be sufficiently large to support k NN search for arbitrary $k \leq K$.

Definition 2.6 (Ranked KNN Graph). Given a dataset $D \subseteq \mathbb{R}^d$ and an integer K , the ranked KNN graph $G_{\text{KNN}} = (V, E)$ is a directed graph with vertex set $V = D$. Each point $o \in V$ stores a **ranked KNN list** of its K nearest neighbors and their distances: $G_{\text{KNN}}[o] = \{(v_1, \delta(o, v_1)), \dots, (v_K, \delta(o, v_K))\}$, where $\delta(o, v_1) \leq \dots \leq \delta(o, v_K)$. A directed edge $o \rightarrow v_j$ exists if v_j is the j -th nearest neighbor of o , i.e., $G_{\text{KNN}}[o, j] = v_j$. For simplicity, we write $G_{\text{KNN}}[o, j] = v_j$, although the actual entry is $(v_j, \delta(o, v_j))$.

The ranked G_{KNN} captures nearest-neighbor relationships from the perspective of each data point in D . Transposing these relationships yields reverse-neighbor lists, which record, for each point, the points in D that regard it as one of their top- K nearest neighbors. Consequently, the Rk NN results of any data point can be obtained directly from its reverse-neighbor list for any $k \leq K$.

Definition 2.7 (Reverse-Neighbor List). Given a ranked G_{KNN} , the **reverse-neighbor list** of a point $o \in D$, denoted by $R[o]$, is defined as $R[o] = \{(v, j) \mid v \in D, G_{\text{KNN}}[v, j] = o, 1 \leq j \leq K\}$, where (v, j) indicates that o is the j -th nearest neighbor of v .

Index construction. Exact construction of a ranked KNN graph requires $O(|D|^2)$ distance computations, since the K -nearest neighbors must be identified for every point in D . To improve efficiency, NNDescend [13] iteratively refines the KNN list of each point based on the results of its current neighbors. Algorithm 1 shows how to create the ranked KNN graph using NNDescend. It initializes each

Algorithm 1: Ranked KNN Graph Construction

Input: Dataset D , integer K
Output: Ranked KNN graph G_{KNN}

```

1 Initialize  $G_{KNN}[o]$  with  $K$  random vectors for each  $o \in D$ ;
2 repeat
3   Create  $R[o] \leftarrow \{x \mid o \in G_{KNN}[x]\}$  for each  $o \in D$ ;
4   for each  $o \in D$  do  $N[o] \leftarrow G_{KNN}[o] \cup R[o]$ ;
5   for each  $o \in D$  do
6     for each neighbor pair  $(u, v)$  in  $N[o]$  do
7       Compute  $d = \delta(u, v)$ ;
8       Insert  $(v, d)$  into  $G_{KNN}[u]$  if improved;
9       Insert  $(u, d)$  into  $G_{KNN}[v]$  if improved;
10 until no neighbor list changes;
```

KNN list $G_{KNN}[o]$ with K random vectors (Line 1). In each iteration, it merges the KNN list $G_{KNN}[o]$ and reverse-neighbor list of each $o \in D$ into a merged set (Lines 3–4). It then computes the exact distance $\delta(u, v)$ for each neighbor pair (u, v) in the merged set and uses it to update KNN lists of both u and v (Lines 5–9). This process repeats until no list changes (Line 10). Specifically, (v, d) updates $G_{KNN}[u]$ if $d = \delta(u, v)$ is smaller than the distance from u to at least one neighbor currently in $G_{KNN}[u]$. For simplicity, we omit several optimizations, such as neighbor sampling and lazy updates.

Navigation Graph. Ranked KNN graphs mainly connect each point to its local neighbors and therefore lack long-range edges. As a result, greedy graph traversal from a random entry point can easily become trapped in local optima [21]. To address this limitation, navigation graphs [16], such as the Hierarchical Navigable Small World (HNSW) graph [42], have been proposed.

Definition 2.8 (HNSW Graph). Given a dataset $D \subset \mathbb{R}^d$, an HNSW graph is an L -layer directed proximity graph $G_{HNSW} = \{G^{(0)}, G^{(1)}, \dots, G^{(L)}\}$, where each layer $G^{(\ell)} = (V^{(\ell)}, E^{(\ell)})$ is defined over a subset $V^{(\ell)} \subseteq D$, satisfying $V^{(L)} \subseteq V^{(L-1)} \subseteq \dots \subseteq V^{(0)} = D$. Each point $o \in D$ is assigned a maximum level $\lambda(o)$ and appears in all layers $G^{(0)}, \dots, G^{(\lambda(o))}$. At each layer $\ell \leq \lambda(o)$, point o maintains a bounded-sized neighbor list $G^{(\ell)}[o] = \{v_1, \dots, v_M\}$, where each $v_i \in V^{(\ell)}$ is selected using proximity-based pruning heuristics. A directed edge $o \rightarrow v_i$ exists in layer $G^{(\ell)}$ if $v_i \in G^{(\ell)}[o]$.

Search on HNSW graph. Given a query q , an HNSW graph G_{HNSW} , the target result size k , and a beam width ef , search on the HNSW index proceeds from the top layer to the bottom layer. Starting from the global entry point in the top layer, the algorithm first performs greedy search on each upper layer with beam width $ef = 1$, so that only the closest candidate is retained and used as the entry point for the next lower layer. After reaching the bottom layer $G^{(0)}$, the algorithm performs beam search with a larger beam width $ef > 1$ to retrieve the final k NN result for the query vector q .

The search within each layer $G^{(\ell)}$ follows Algorithm 2. Given the current entry point ep , the algorithm initializes a candidate heap Q and a result heap $W[q]$ with ep (Lines 1–2), and marks it as visited (Line 3). It then repeatedly extracts the closest vertex v from Q (Line 5) and compares it with the farthest vertex u in $W[q]$ (Line 6). If v is farther from q than u , the search stops because no remaining candidate can improve the current result (Line 7). Otherwise, the algorithm scans each unvisited neighbor $o \in G^{(\ell)}[v]$ (Line 8), marks it as visited (Line 9), and inserts it into both Q and $W[q]$ if $W[q]$ has fewer than k vertices or o is closer to q than the current farthest candidate (Lines 10–11). If $W[q]$ or Q exceeds its size limit, the

Algorithm 2: GRAPH-SEARCH(G, q, k, ep, ef)

Input: Graph G , query vector q , target number of results k , entry point ep , beam width ef
Output: Candidate set $W[q]$ of size k

```

1 Initialize min-heap  $Q \leftarrow \{ep\}$  ordered by  $\delta(q, \cdot)$ ;
2 Initialize max-heap  $W[q] \leftarrow \{ep\}$  ordered by  $\delta(q, \cdot)$ ;
3 Mark  $ep$  as visited;
4 while  $Q \neq \emptyset$  do
5   Extract the closest vertex  $v$  from  $Q$ ;
6   Let  $u$  be the farthest vertex in  $W[q]$ ;
7   if  $\delta(q, v) > \delta(q, u)$  then break;
8   for each unvisited neighbor  $o \in G[v]$  do
9     Mark  $o$  as visited;
10    if  $|W[q]| < k$  or  $\delta(q, o) < \delta(q, u)$  then
11      Insert  $o$  into both  $Q$  and  $W[q]$ ;
12    if  $|W[q]| > k$  then
13      Remove the farthest vertex from  $W[q]$ ;
14    if  $|Q| > ef$  then
15      Remove the farthest vertex from  $Q$ ;
16 return  $W[q]$ ;
```

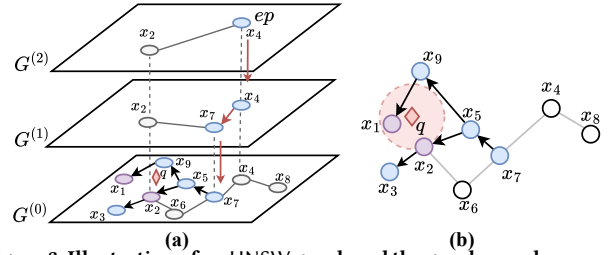


Figure 3: Illustration of an HNSW graph and the graph search process. farthest vertex is removed (Lines 12–15). The search returns $W[q]$ as the result (Line 16).

Example 2.9. Fig. 3(a) illustrates an HNSW graph with three layers, where $G^{(2)}$ is the top layer and $G^{(0)}$ is the bottom layer. Given a query point q , the top-down routing phase starts from the global entry point $ep = x_4$ and performs greedy search ($ef = 1$) on each layer. The search progressively descends through the hierarchy until reaching the bottom layer, where x_7 is identified as the entry point. Fig. 3(b) illustrates the bottom-layer beam search ($ef = 2$). Starting from x_7 , the search visits x_5 and inserts x_2 and x_9 into the candidate queue. Since $\delta(q, x_2) < \delta(q, x_9)$, x_2 is expanded first, followed by x_9 , which discovers x_1 . The final 2NN result is $W[q] = \{x_1, x_2\}$ (shaded in purple).

3 PROBLEM ANALYSIS

We summarize existing methods for Rk NN search in Section 3.1, and then analyze their two limitations in Section 3.2. This analysis motivates our method presented in Section 4. For simplicity, we use the term Rk NN search to refer to approximate Rk NN search throughout the remainder of the paper unless otherwise specified.

3.1 Existing Solutions

The main challenge in answering the Rk NN search problem lies in two aspects. First, unlike k NN search, where the result points are directly close to the query vector q , the points in the Rk NN

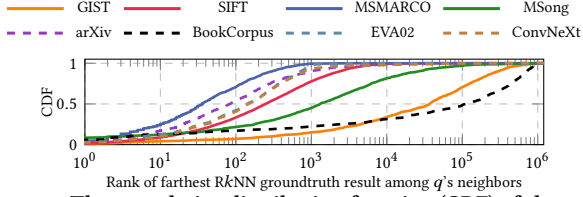


Figure 4: The cumulative distribution function (CDF) of the maximum nearest-neighbor rank (among the ground-truth RkNN results) for queries on all eight datasets listed in Table 3, with $k = 10$.

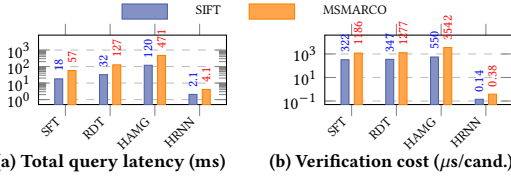


Figure 5: Comparison of query performance for approximate RkNN search with $k = 10$ at Recall@10 = 0.99. (a) shows the total query latency of HRNN and the baselines (SFT, RDT, and HAMG). (b) reports the per-candidate verification cost of HRNN and the baselines.

set do not necessarily have explicit proximity relationships with q . Specifically, RkNN search is defined from the perspective of data points $o \in D$: we need to determine whether q belongs to the k NN set of point o by checking the k NN-radius of o . Second, while k NN search always returns exactly k vectors, the number of reverse k NN results for a query vector is unknown in advance.

Filter-and-Verification Framework. Directly applying the definition requires scanning every data point $o \in D$ and checking whether $\delta(q, o) \leq r_k(o)$, which is prohibitively expensive at scale. Existing methods therefore adopt a filter-and-verification framework: Instead of examining all points in D , these methods first reduce the search space to a candidate set $C \subseteq D$, and then verify whether each candidate point belongs to the RkNN result set of the query vector q . To obtain a compact candidate set C , existing methods commonly assume that **data points close to q are more likely to regard q as one of their k nearest neighbors, and thus are more likely to belong to the RkNN result set of q .** Based on this assumption, methods such as SFT [48] and RDT [6] first perform nearest-neighbor search and then examine nearby neighbors as candidates until a stopping condition is satisfied.

The State-of-the-art. More recently, HAMG [50] was proposed following the same intuition. A key insight of HAMG is that, on the Monotonic Relative Neighborhood (MRN) graph [4, 8, 13, 15, 16, 21, 25, 26, 33, 38, 40, 42, 46, 55, 57, 60, 61, 67], the RkNN results of a query vector q are guaranteed to lie within the k -hop neighborhood of q . Based on this property, HAMG treats these nearby graph neighbors as the candidate set C . Specifically, HAMG adopts the hierarchical structure of the HNSW graph in Definition 2.8, while modifying the bottom layer $G^{(0)}$ to better approximate the MRN graph structure. Using this adapted HNSW graph, HAMG retrieves the k -hop neighbors of q as candidates and then verifies whether $\delta(q, o) \leq r_k(o)$ for each candidate $o \in C$ to determine if o belongs to the RkNN set of q . More details can be found in [50].

3.2 Limitation Analysis

Existing methods avoid scanning the entire dataset D to retrieve the RkNN results of a query vector q . Yet, they still suffer from two major limitations that reduce their efficiency on large datasets.

Limitation 1: The mismatch between RkNN and nearby neighbors. Existing methods assume that nearby neighbors of q —i.e., the top- k' neighbors in SFT and RDT, or the k -hop graph neighbors in HAMG—contain all RkNN results. While this assumption generally holds, retrieving a sufficiently large neighborhood can incur substantial query latency in practice. Even for the state-of-the-art HAMG, the number of k -hop graph neighbors can become extremely large due to the graph expansion effect. Thus, substantial computation spent exploring nearby neighbors is unnecessary.

To illustrate this, Fig. 1 shows that a data point far from the query q can still be an RkNN result, requiring exploration far beyond q 's nearby neighbors. We further quantify this limitation in Fig. 4 across all datasets listed in Table 3 with $k = 10$. For each query q , we compute its exact RkNN results and record the nearest-neighbor rank of each result o , where rank k' indicates that o is the k' -th nearest neighbor of q . We then take the **maximum rank k''** among all RkNN results, such that the top- k'' nearest neighbors of q contain all RkNN results. We report the CDF of k'' over queries with nonempty results. The results reveal substantial non-locality: on SIFT, over 50% of queries require examining more than 250 neighbors to retrieve all RkNN results, while on GIST and BookCorpus, this number can reach 10^6 . These results demonstrate the inefficiency of relying solely on nearby neighbors for RkNN search.

Limitation 2: Costly online k NN-radius computation. The first limitation results in a candidate set C substantially larger than the true result set. More importantly, each candidate $o \in C$ must be verified against its k NN-radius $r_k(o)$. Existing methods typically obtain $r_k(o)$ by performing an online k NN search centered at o during verification [6, 48, 50, 68]. Since C is unknown in advance and often large, computing $r_k(o)$ online for every candidate incurs substantial query overhead. To quantify this limitation, we compare our proposed method HRNN with SFT, RDT, and HAMG on the datasets SIFT and MSMARCO in Fig. 5. HRNN achieves up to an order-of-magnitude lower query latency (Fig. 5(a)), as existing methods incur substantially higher verification costs from online k NN-radius computation for each candidate (Fig. 5(b)).

4 THE PROPOSED METHOD

We introduce our new index structure in Section 4.1, followed by the query processing algorithm in Section 4.2. Then, Section 4.3 describes the index construction procedure, while Section 4.4 presents index maintenance under data point insertions and deletions.

4.1 Index Structure

We now present the key ideas for addressing the two limitations of existing RkNN methods.

Idea 1. To address Limitation 1, we revisit the relationship between k NN and RkNN search. Existing methods treat the nearest neighbors of the query as candidates for RkNN search. Yet, our analysis shows that true RkNN results are often not among the nearby neighbors. Instead, we make a different assumption: **A query's RkNN results can often be found by examining the reverse nearest**

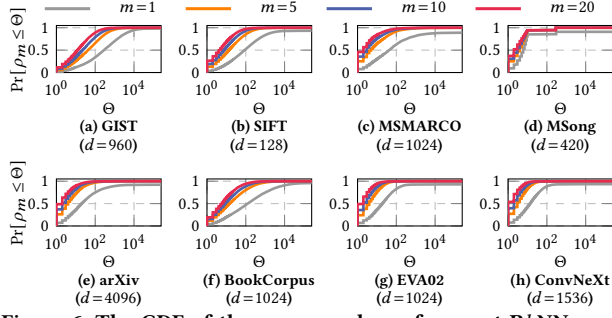


Figure 6: The CDF of the proxy rank ρ_m for exact RkNN results ($k = 10$) with $m \in \{1, 5, 10, 20\}$ on all eight datasets listed in Table 3.

neighbors of nearby points, which we call proxy points. To validate this assumption, we select the top- m nearest neighbors of each query q as **proxies** (e.g., $m = 5$). For each exact RkNN result x of query q (with $k = 10$), we define its *proxy rank* $\rho_m(q, x)$ as the minimum rank of these proxies in x 's nearest-neighbor list. A proxy rank of r indicates that some proxy ranks within the top- r neighbors of x , and thus x can be recovered from that proxy's RrNN results. Fig. 6 shows the CDF of proxy ranks across all datasets in Table 3. About half of the results have proxy ranks below 50, suggesting that many results can be recovered from the proxies' reverse nearest neighbors using a relatively small rank threshold.

This assumption follows from spatial proximity: as q and its nearby neighbors lie close to each other, they are likely to fall within the k NN-radius of the same point x . Thus, if x is an RkNN of q 's nearby neighbors, it is also likely to appear in the RkNN set of q . Based on this intuition, we first retrieve a set of nearby points as *proxies*. Rather than treating these proxies as RkNN candidates, we leverage their reverse-neighbor lists, which provide their RkNN results to generate candidates for the query. This design avoids excessive neighborhood expansion while keeping high accuracy.

Idea 2. To address Limitation 2, we eliminate the expensive online computation of k NN-radius during verification. Recall that the k NN-radius of $o \in D$ is simply the distance between o and its k -th nearest neighbor. Thus, if we materialize a ranked KNN list for each vector and store the corresponding distances in ascending order, the k NN-radius for any value of $k \leq K$ can be retrieved directly through a lookup. This transforms radius computation from an expensive online search into a lightweight index-access operation.

Index Design. To realize the above intuitions, the index must support three operations: (1) efficiently retrieving nearby neighbors of a query vector as proxies, (2) obtaining the reverse-neighbor lists of proxy points to generate RkNN candidates for the query, and (3) maintaining ranked KNN lists for all points in the dataset so that k NN-radius can be accessed efficiently during verification.

Designing a single index that meets all these requirements is challenging. An HNSW graph efficiently retrieves nearest neighbors as proxies but does not maintain ranked KNN lists. Conversely, a ranked KNN graph explicitly stores these lists but is inefficient for query-time nearest-neighbor search. Moreover, neither structure directly supports efficient retrieval of RkNN results for data points. We therefore propose HRNN, a hybrid index that integrates an HNSW graph, a ranked KNN graph, and reverse-neighbor lists to support RkNN search.

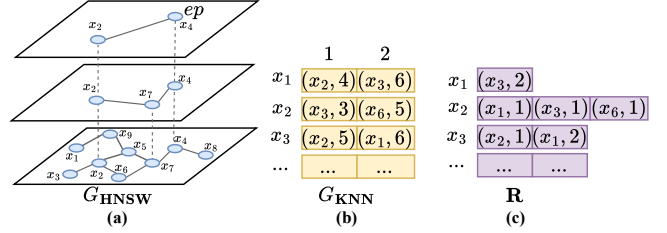


Figure 7: An example of the HRNN index structure.

Definition 4.1 (HRNN Index). Given a dataset D and a user-defined parameter K , the HRNN index is defined as $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$, where:

- (1) G_{HNSW} is an HNSW graph constructed over D (Definition 2.8);
- (2) G_{KNN} is a ranked KNN graph constructed over D with parameter K (Definition 2.6);

- (3) $\mathbf{R}$ denotes reverse-neighbor lists derived from G_{KNN} . For each point $o \in D$, its reverse-neighbor list $\mathbf{R}[o]$ stores all pairs (v, j) such that o is the j -th nearest neighbor of v in G_{KNN} (Definition 2.7).

Example 4.2. Fig. 7 shows the components of HRNN. Fig. 7(a) shows the HNSW graph. Fig. 7(b) presents the ranked KNN graph, where each node o maintains a list of K -nearest neighbors ($K = 2$), and each entry is denoted as $(v_i, \delta(o, v_i))$. For instance, x_2 is the 1NN of x_3 with $\delta(x_3, x_2) = 5$. Fig. 7(c) shows the reverse-neighbor lists derived from G_{KNN} . For example, since x_3 is the 1NN of x_2 and the 2NN of x_1 , its reverse-neighbor list is $\mathbf{R}[x_3] = \{(x_2, 1), (x_1, 2)\}$, which records these reverse-neighbor relationships.

THEOREM 4.3. The HRNN index requires $O(|D| \cdot M \cdot L + |D| \cdot K)$ space, where L is the HNSW layer number, M is the maximum degree of G_{HNSW} , and K is the number of stored ranked neighbors per point.

PROOF. The HRNN index consists of three components. First, the HNSW graph stores at most M outgoing neighbors for each vector at each layer. Since there are at most $|D|$ vectors and L layers, the space required by the HNSW graph is bounded by $O(|D| \cdot M \cdot L)$. Second, the ranked KNN graph stores K ranked neighbors for each vector, requiring $O(|D| \cdot K)$ space. Third, the reverse-neighbor lists are obtained by transposing the ranked KNN graph. Each ranked neighbor entry generates exactly one reverse posting, yielding $|D| \cdot K$ reverse postings in total and therefore requiring $O(|D| \cdot K)$ space. Summing the three components gives $O(|D| \cdot M \cdot L + 2|D| \cdot K) = O(|D| \cdot M \cdot L + |D| \cdot K)$. $\square$

Remark. The index stores exactly K nearest neighbors for each point in G_{KNN} and thus supports RkNN queries only with $k \leq K$. We set K sufficiently large (e.g., 100) to cover typical query settings.

4.2 Query Processing

We now describe how to answer an approximate RkNN query for a query vector q using the proposed HRNN index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$. If the query vector q were part of the indexed dataset D , its reverse-neighbor list $\mathbf{R}[q]$ would directly reveal the points whose KNN neighborhoods contain q . However, in practice, query vectors are typically not part of the indexed dataset. To address this issue, HRNN first retrieves a set of nearby points as *proxies* and uses their reverse-neighbor lists to approximate the unavailable reverse-neighborhood information of q .

Algorithm 3: Approximate RkNN Search

Input: Index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$, query vector q , target parameter k , proxy size m , rank threshold Θ , HNSW search parameters ep and ef_s

Output: ARkNN result set R

```

1  $R \leftarrow \emptyset, C \leftarrow \emptyset;$ 
2  $N_m(q) \leftarrow \text{GRAPH-SEARCH}(G_{\text{HNSW}}, q, m, ep, ef_s);$ 
3 for  $b \in N_m(q)$  do
4   for  $(v, j) \in \mathbf{R}[b]$  in ascending order of  $j$  do
5     if  $j > \Theta$  then break;
6      $C \leftarrow C \cup \{v\};$ 
7 for  $o \in C$  do
8    $v_k \leftarrow G_{\text{KNN}}[o, k];$ 
9    $\hat{r}_k(o) \leftarrow \delta(o, v_k);$ 
10  if  $\delta(q, o) \leq \hat{r}_k(o)$  then  $R \leftarrow R \cup \{o\};$ 
11 return  $R;$ 
```

Filter. Let $N_m(q) = \{b_1, \dots, b_m\}$ denote the approximate m -nearest neighbors of q returned by the HNSW graph. Rather than treating these points as RkNN candidates, HRNN uses them as proxy points to discover candidate points. To avoid scanning excessively large reverse-neighbor lists $\mathbf{R}[b]$ of the proxy point $b \in N_m(q)$, we introduce a rank threshold Θ and only consider objects whose ranks do not exceed Θ . This design is based on the intuition that if the rank of a proxy b with respect to an object v exceeds Θ , then b —and thus the query vector—is unlikely to be within the top- Θ nearest neighbors of v . Hence, v can be safely pruned, as it is unlikely to be an R Θ NN of q . Formally, the candidate set of query q is defined as

$$C_{m,\Theta}(q) = \bigcup_{b \in N_m(q)} \{v \mid (v, j) \in \mathbf{R}[b], j \leq \Theta\}. \quad (1)$$

Note that $\mathbf{K}$ denotes the number of neighbors materialized in the ranked KNN graph, whereas $\Theta \leq \mathbf{K}$ is a query-time parameter that determines when to stop scanning the reverse-neighbor list $\mathbf{R}[b]$ of proxy b : the scan stops upon encountering an entry $(v, j) \in \mathbf{R}[b]$ with rank $j > \Theta$. A larger Θ increases candidate coverage and potentially improves accuracy, but incurs higher costs.

Verification. After candidate generation, HRNN verifies whether each candidate $o \in C_{m,\Theta}(q)$ belongs to the final RkNN result set of q . Existing methods typically compute the k NN-radius of each candidate online, which incurs substantial overhead. In contrast, HRNN leverages the pre-materialized ranked KNN graph G_{KNN} . Since the graph stores the top- $\mathbf{K}$ nearest neighbors of every point, when $\mathbf{K} \geq k$, the k -th nearest neighbor of o can be directly retrieved from its KNN list $G_{\text{KNN}}[o]$. Let v_k denote this neighbor. The corresponding distance $\hat{r}_k(o) = \delta(o, v_k)$ serves as an estimate of the k NN-radius of o . Using this pre-materialized radius, a candidate o is accepted if $\delta(q, o) \leq \hat{r}_k(o)$. This verification process reduces candidate validation to a single distance comparison.

Algorithm. Algorithm 3 presents the procedure for answering an approximate RkNN query for a query vector q using the hybrid index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$. The algorithm first performs an approximate m -nearest-neighbor search on G_{HNSW} to retrieve a set of proxy points $N_m(q)$ that are close to q (Line 2). The details of the graph search procedure are provided in Algorithm 2. For each proxy point $b \in N_m(q)$, the algorithm scans its reverse-neighbor list $\mathbf{R}[b]$ in ascending rank order (Lines 3–4). Only vertices whose ranks do not exceed the threshold Θ are inserted into the candidate set C (Lines 5–6). This proxy-guided lookup process

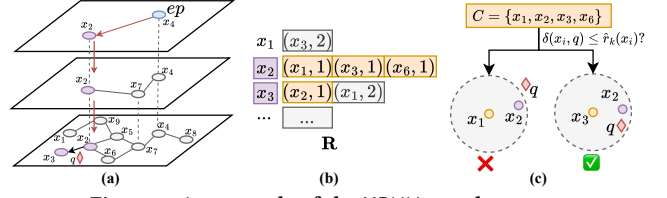


Figure 8: An example of the HRNN search process.

generates candidate RkNN results without expensive neighborhood expansion. The algorithm then verifies each candidate $o \in C$ (Line 7). Specifically, it retrieves the k -th nearest neighbor v_k of o from its KNN list $G_{\text{KNN}}[o]$ and obtains the corresponding estimated k NN-radius $\hat{r}_k(o) = \delta(o, v_k)$ (Lines 8–9). A candidate is accepted if $\delta(q, o) \leq \hat{r}_k(o)$ (Line 10). All verified candidates are added to the result set R , which is returned as the final RkNN result (Line 11).

Example 4.4. Fig. 8 illustrates HRNN query processing for a query vector q , with $m = 2$ and $\Theta = 1$. The search first traverses G_{HNSW} and retrieves x_2 and x_3 as proxy points, as shown in Fig. 8(a). As illustrated in Fig. 8(b), HRNN then scans the reverse-neighbor lists of the proxies in ascending rank order and inserts objects whose rank does not exceed Θ into the candidate set C . In this example, x_1, x_2, x_3 , and x_6 are added to C . Finally, each candidate is verified by checking whether q falls within its k NN-radius, which is obtained directly from G_{KNN} . Candidates satisfying this condition (e.g., x_3) are returned as RkNN results.

THEOREM 4.5. *Given a query q , Algorithm 3 answers an RkNN query in $O(C_H^{\text{qry}}(q) + s(q) + d \cdot u(q))$ time, where $C_H^{\text{qry}}(q)$ is the cost of the HNSW search for retrieving the m proxies, $s(q)$ is the number of scanned reverse-neighbor objects, and $u(q)$ is the number of distinct candidates after deduplication, i.e., $|C_{m,\Theta}(q)|$ in Equation 1.*

PROOF. Retrieving $N_m(q)$ via HNSW incurs cost $C_H^{\text{qry}}(q)$. We keep this term explicit because HNSW has no proven query-complexity bound for general datasets; prior studies typically model its query cost as $O(\log |D|)$ or observe approximately logarithmic scaling under their evaluated workloads and parameter settings [36, 38, 42, 44, 58, 60]. Candidate generation scans $s(q)$ reverse-neighbor entries, taking $O(s(q))$ time. After deduplication, verifying $u(q)$ candidates requires one exact distance computation and one constant-time radius lookup per candidate, costing $O(d \cdot u(q))$. Summing these costs gives the stated bound. $\square$

Theoretical Analysis. The intuition behind our technique—using the RkNN results of nearby proxies to retrieve those of the query vector—is empirically supported by Fig. 6. We further provide the following theoretical analysis to justify this intuition.

When does our method work? We first characterize when our method retrieves a true RkNN result. For each data point $o \in D$, let $r_j(o)$ denote the distance from o to its j -th nearest neighbor in $D \setminus \{o\}$. We assume that all pairwise distances are distinct (ties can be broken by an arbitrarily small perturbation) and that each $o \in D$ stores its exact $\mathbf{K}$ nearest neighbors in $G_{\text{KNN}}[o]$. We further require $k \leq \mathbf{K}$ and $k - 1 \leq \Theta \leq \mathbf{K}$ to ensure exact verification and sufficient candidate retrieval, respectively.

We next characterize the search process, as illustrated in Fig. 9. Let q be a query vector and $o \in D$ a true RkNN result of q , meaning that q is among the k nearest neighbors of o . The search uses the

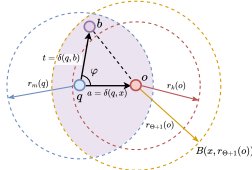


Figure 9: Illustration of how HRNN retrieves a true RkNN result o of query q through a proxy b .

m nearest neighbors of q , denoted by $N_m(q)$, as proxies to discover such results. Consider a proxy $b \in N_m(q)$: although b is selected for its proximity to q , whether it can discover o depends on its rank among the KNN list entries of o . Suppose $(o, j) \in R[b]$, meaning that b is the j -th nearest neighbor of o . Since the search scans only entries with rank at most Θ in each reverse-neighbor list, o can be discovered through b if and only if $j \leq \Theta$, equivalently, $\delta(o, b) < r_{\Theta+1}(o)$, where $r_{\Theta+1}(o)$ is the distance from o to its $(\Theta+1)$ -th nearest neighbor. Geometrically, this means that b lies in the open ball centered at o with radius $r_{\Theta+1}(o)$. We call this ball the *witness ball* of o , and each data point within it other than o a *witness* of o . Thus, the search discovers o through its proxies if and only if the proxy set $N_m(q)$ contains a witness of o , that is, $N_m(q) \cap B(o, r_{\Theta+1}(o)) \neq \emptyset$.

THEOREM 4.6. *Given a query q , a true RkNN result $o \in A_k(q)$ is returned by Algorithm 3 if and only if the proxy set $N_m(q)$ contains a witness of o , i.e., $N_m(q) \cap B(o, r_{\Theta+1}(o)) \neq \emptyset$.*

PROOF. The case $b = o$ is trivial, so we consider $b \neq o$. If $b \in N_m(q) \cap B(o, r_{\Theta+1}(o))$, then $R[b]$ contains a posting (o, j) with $j \leq \Theta$. Thus, $o \in C_{m, \Theta}(q)$ and is correctly returned after verification. Conversely, if no proxy lies in $B(o, r_{\Theta+1}(o))$, no proxy's reverse-neighbor list contains a posting (o, j) with $j \leq \Theta$. Hence, o cannot be discovered or returned. $\square$

Theorem 4.6 shows that an RkNN result o of q is returned if at least one witness of o is selected as a proxy. We now characterize when a proxy b is such a witness, that is, when b lies in the witness ball of o . For a fixed query-result pair (q, o) , let $a = \delta(q, o)$ be their distance and $R = r_{\Theta+1}(o)$ be the radius of the witness ball centered at o . We describe the position of b relative to q and o using two quantities: its distance from the query, $t = \delta(q, b)$, and the angle $\varphi \in [0, \pi]$ between the vectors $b - q$ and $o - q$. These quantities allow us to express the witness condition in terms of t and φ .

LEMMA 4.7. *Given a query q and $o \in A_k(q)$, a proxy $b \in D$ lies in the witness ball $B(o, r_{\Theta+1}(o))$ if and only if $0 \leq t < g(\varphi)$, where*

$$g(\varphi) = a \cos \varphi + \sqrt{R^2 - a^2 \sin^2 \varphi} \quad (2)$$

is nonincreasing on $[0, \pi]$. In particular, b is a witness of o if either (i) $t < g(\pi) = R - a$, regardless of its direction, or (ii) $\langle b - q, o - q \rangle \geq 0$ and $t < g(\pi/2) = \sqrt{R^2 - a^2}$.

PROOF. By the law of cosines, $\delta(o, b)^2 = t^2 + a^2 - 2ta \cos \varphi$. Thus, $b \in B(o, r_{\Theta+1}(o))$ iff $t^2 - 2ta \cos \varphi + a^2 - R^2 < 0$, whose roots are $a \cos \varphi \pm \sqrt{R^2 - a^2 \sin^2 \varphi}$. Since $R \geq a$ (which holds since $k \leq \Theta + 1$ and o is a true RkNN result), the smaller root is nonpositive; hence, for $t \geq 0$, the condition is equivalent to $t < g(\varphi)$. For fixed t and a , $\delta(o, b)$ is nondecreasing in φ , implying that g is nonincreasing. For (i), $t < g(\pi) \leq g(\varphi)$ for any φ . For (ii), $\langle b - q, o - q \rangle \geq 0$ implies $\varphi \leq \pi/2$, and thus $t < g(\pi/2) \leq g(\varphi)$. $\square$

Geometrically, $g(\varphi)$ is the distance from q to the boundary of $B(o, R)$ in direction φ . Condition (i) ensures that any proxy at distance $t < g(\pi) = R - a$ from q can discover o , regardless of direction. Condition (ii) permits a larger distance, $t < g(\pi/2) = \sqrt{R^2 - a^2}$, when the proxy lies on the side of q facing o , i.e., $\langle b - q, o - q \rangle \geq 0$.

Probabilistic analysis. The above analysis characterizes when our method retrieves a true RkNN result o of query q . We next provide a probabilistic analysis to quantify the likelihood of successful retrieval. Following common random-geometric assumptions for nearest-neighbor search, we model the nearest neighbors of a point as uniformly distributed within a local ball [13, 28, 46]. A weaker form assumes that their normalized directions are independent and uniformly distributed on the unit sphere [64, 75]. Under this uniform-direction assumption, a random neighbor of a data point lies on either side of a hyperplane through the center with probability $1/2$ [15, 28].

THEOREM 4.8. *Given a query q , let $N_m(q) = \{b_1, \dots, b_m\}$ be its proxy set. Conditioned on the proxy distances $\delta(q, b_1), \dots, \delta(q, b_m)$, assume that the unit directions $(b_j - q)/\delta(q, b_j)$ are independent and uniformly distributed on the unit hypersphere. For a true RkNN result o of q , let*

$$h(o) = \left| \left\{ j \leq m : \delta(q, b_j) < \sqrt{R^2 - a^2} \right\} \right|$$

denote the number of proxies satisfying the distance condition for discovering o . Then, Algorithm 3 returns o with probability at least $1 - 2^{-h(o)}$. By the union bound, the returned set equals the exact RkNN result set $A_k(q)$ with probability at least $1 - \sum_{o \in A_k(q)} 2^{-h(o)}$.

PROOF. We bound the probability that a true RkNN result o of q has no witness in the proxy set. Each eligible proxy b_j satisfies $\delta(q, b_j) < g(\pi/2)$ and, by Condition (ii) of Lemma 4.7, lies in $B(o, r_{\Theta+1}(o))$ whenever $\langle b_j - q, o - q \rangle \geq 0$. Under the uniform-direction assumption, this event occurs independently with probability $1/2$ for each b_j . Thus, missing o requires all $h(o)$ eligible proxies to fail, which occurs with probability at most $2^{-h(o)}$. $\square$

Implications of the Analysis. Theorem 4.8 assumes that, conditioned on their distances from the query, proxy directions are independent and uniformly distributed. Each eligible proxy then lies in the half-space facing o with probability $1/2$, allowing it to discover o under Condition (ii) of Lemma 4.7. Thus, each additional eligible proxy halves the upper bound $2^{-h(o)}$ on the probability of missing o , motivating the use of multiple proxies. In contrast, Theorem 4.6 and Lemma 4.7 do not require this distributional assumption. The analysis highlights two parameters: the cutoff Θ and the proxy count m . Increasing Θ enlarges the witness ball $B(o, r_{\Theta+1}(o))$, while increasing m adds proxies that may fall within it. Under the assumptions of Theorem 4.6, the returned set cannot shrink as Θ increases with fixed proxies, or as m increases with nested proxy sets. As shown in Section 5, moderate values of m and Θ achieve high recall and precision across all evaluated datasets, empirically supporting our analysis.

Remark on kNN-radius. The previous analysis characterizes when a true RkNN result o of query q can be discovered. Another key step is verification, where each candidate o is verified by comparing $\delta(q, o)$ with the kNN-radius $r_k(o)$ of the data point o . Recall that Line 10

Algorithm 4: HRNN Index Construction

Input: Dataset $D \subset \mathbb{R}^d$; HNSW parameters M and ef_c ; ranked KNN size K ; NNDescent iterations T

Output: Index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$

// Phase 1: Build the HNSW graph

- 1 $(G_{\text{HNSW}}, W) \leftarrow \text{BUILD-HNSW}(D, M, ef_c)$;
- // Phase 2: Construct the ranked KNN graph
- 2 Initialize $G_{\text{KNN}}[o]$ using $W[o]$ for each $o \in D$;
- 3 Execute Lines 2–10 of Algorithm 1 to build G_{KNN} ;
- // Phase 3: Construct reverse-neighbor lists
- 4 Initialize $\mathbf{R}[o] \leftarrow \emptyset$ for all $o \in D$;
- 5 **for each** $o \in D$ **do**
- 6 **for** $j \leftarrow 1$ **to** K **do**
- 7 $v \leftarrow G_{\text{KNN}}[o, j]$;
- 8 Append (o, j) to $\mathbf{R}[v]$;
- 9 **for each** $o \in D$ **do**
- 10 Sort $\mathbf{R}[o]$ in ascending order of rank j ;
- 11 **return** $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$;

of Algorithm 3 instead uses the estimated radius $\hat{r}_k(o)$ materialized in our index. We next show that using $\hat{r}_k(o)$ does not cause any true RkNN result to be rejected.

PROPOSITION 1. *Suppose $G_{\text{KNN}}[o]$ contains at least k distinct points ordered by their distances from o . Then $\hat{r}_k(o) \geq r_k(o)$. Thus, every true RkNN result o that enters $C_{m,\Theta}(q)$ is correctly reported.*

PROOF. The exact radius $r_k(o)$ is the k -th smallest distance from o among all data points other than o . Since $G_{\text{KNN}}[o]$ contains a subset of these points, its k -th smallest distance cannot be smaller than $r_k(o)$. Thus, $\hat{r}_k(o) \geq r_k(o)$. For any $o \in A_k(q)$, we have $\delta(q, o) \leq r_k(o) \leq \hat{r}_k(o)$, so the verification accepts o . $\square$

4.3 Index Construction

Constructing the HRNN index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$ is challenging because it consists of three coupled structures: the HNSW graph, the ranked KNN graph, and the reverse-neighbor lists. A straightforward approach is to construct them independently, but this would incur redundant neighborhood exploration and repeated nearest-neighbor computations.

To avoid this overhead, HRNN adopts a unified construction pipeline. It first builds the HNSW graph and records the neighbors discovered during the bottom-layer searches for each inserted point. Since these neighbors already approximate the local neighborhood of the point, they provide high-quality seeds for initializing its ranked KNN list. HRNN then applies NNDescent to refine these initial neighborhoods and construct a high-quality ranked KNN graph. Compared with random initialization, this strategy improves convergence and reduces construction cost. Finally, the reverse-neighbor lists are derived by transposing the ranked KNN graph: if a point u is the j -th nearest neighbor of a point v , the pair (v, j) is inserted into the reverse-neighbor list of u . In this way, HRNN constructs all three index structures efficiently while maximizing the reuse of neighborhood information across different index views.

Algorithm. Algorithm 4 follows this pipeline in three phases. First, it builds G_{HNSW} while recording the bottom-layer search results $W[o]$ for each point $o \in D$ (Line 1). Second, it initializes $G_{\text{KNN}}[o]$ with $W[o]$ and refines it using Lines 2–10 of Algorithm 1 (Lines 2–3). Finally, it transposes G_{KNN} into reverse-neighbor lists: if $v =$

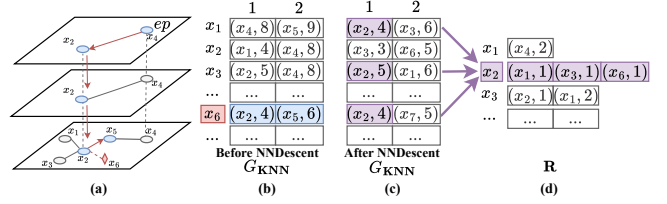


Figure 10: An example of HRNN index construction.

$G_{\text{KNN}}[o, j]$, then (o, j) is inserted into $\mathbf{R}[v]$ (Lines 4–8). Each $\mathbf{R}[v]$ is then sorted by rank (Lines 9–10) to satisfy its definition.

Example 4.9. Fig. 10 illustrates the construction process of the HRNN index. Fig. 10(a) shows the search performed when inserting a new point x_6 into G_{HNSW} , and x_6 is connected to x_2 after pruning the search results. Other points are inserted similarly. The search result $[x_2, x_3]$ is recorded and used to initialize the ranked KNN graph with $K = 2$, as shown in Fig. 10(b). Fig. 10(c) shows how NNDescent refines the ranked KNN graph. For example, in x_6 's neighbor list, the initial 2NN x_3 is replaced by the closer neighbor x_7 . Finally, Fig. 10(d) constructs the reverse-neighbor lists by scanning the refined ranked KNN graph. For instance, since x_2 appears as the 1NN of x_1 , x_3 , and x_6 , its reverse-neighbor list is $\mathbf{R}[x_2] = [(x_1, 1), (x_3, 1), (x_6, 1)]$.

THEOREM 4.10. *Algorithm 4 constructs the HRNN index in $O(C_H^{\text{bld}}(D) + T|D|K^2d + |D|K)$ time, where $C_H^{\text{bld}}(D)$ is the total cost of inserting all $|D|$ vectors into HNSW, d is the vector dimension, K is the ranked neighbor-list size, and T is the number of NNDescent refinement iterations.*

PROOF. Algorithm 4 consists of three phases. First, it constructs the HNSW graph over D by incrementally inserting vectors. We denote the total realized insertion cost by $C_H^{\text{bld}}(D)$. During construction, HNSW also records the bottom-layer search results as initialization seeds for the ranked KNN graph, which does not change the asymptotic cost. Second, HRNN constructs the ranked KNN graph, where each vector maintains K ranked neighbors. The lists are initialized from the HNSW search results and refined using NNDescent. In each iteration, NNDescent performs $O(K^2)$ candidate comparisons per vector, each requiring $O(d)$ time for distance computation. Thus, T refinement iterations cost $O(T|D|K^2d)$. Third, HRNN constructs the reverse-neighbor lists by transposing the ranked KNN graph. Since the graph contains $|D|K$ directed neighbor entries and each produces one reverse posting, this phase costs $O(|D|K)$. Summing the three phases gives $O(C_H^{\text{bld}}(D) + T|D|K^2d + |D|K)$. $\square$

4.4 Index Maintenance

Real-world datasets continually evolve through point insertions and deletions. Rebuilding HRNN after every update is prohibitively expensive, so we develop efficient algorithms for both operations.

Point Insertion. Given a newly inserted data point o_{new} , maintenance must synchronize three structures: insert o_{new} into G_{HNSW} for future proxy retrieval, update KNN lists in G_{KNN} for points whose top- K neighbors include o_{new} , and update the corresponding reverse-neighbor lists in $\mathbf{R}$. We first describe how to identify affected points and then explain how to update the index accordingly.

Exact affected set. The first challenge is identifying data points in D affected by inserting o_{new} . Let $r_K(x)$ denote the distance from x to its current K -th nearest neighbor. The exact affected set is

$$S^*(o_{\text{new}}) = \{x \in D \mid \delta(x, o_{\text{new}}) < r_K(x)\}, \quad (3)$$

where each $x \in S^*(o_{\text{new}})$ requires updates to its ranked KNN list (and the corresponding reverse-neighbor lists of other data points). However, computing $S^*(o_{\text{new}})$ exactly requires checking all points $x \in D$, incurring an impractical $O(|D|)$ scan per insertion.

Approximating affected set. To avoid exhaustive scans, we observe that points affected by inserting o_{new} have o_{new} among their K nearest neighbors. We therefore approximate the affected set via an RkNN-style search on the current index for o_{new} . Specifically, we retrieve nearby proxies $N_{m_u}(o_{\text{new}})$ from the current HNSW and, given an update-time rank threshold $\Theta_u \leq K$, define

$$\hat{S}_u(o_{\text{new}}) = \bigcup_{b \in N_{m_u}(o_{\text{new}})} \{x \mid (x, j) \in R[b], j \leq \Theta_u\}. \quad (4)$$

Thus, $\hat{S}_u(o_{\text{new}})$ contains points having at least one proxy among their top- Θ_u neighbors. Increasing Θ_u improves coverage at the cost of scanning more reverse-neighbor entries.

Maintenance procedure. We next describe how to update the index. We first insert o_{new} into HNSW and initialize $G_{\text{KNN}}[o_{\text{new}}]$ with the search results $W[o_{\text{new}}]$. The reverse-neighbor lists of objects in $G_{\text{KNN}}[o_{\text{new}}]$ are updated accordingly. Then, for each affected object $x \in \hat{S}_u(o_{\text{new}})$, if $\delta(x, o_{\text{new}})$ is smaller than the distance to its current K -th neighbor, we insert o_{new} into $G_{\text{KNN}}[x]$ and evict the farthest neighbor if necessary. We then update the corresponding reverse-neighbor lists by removing obsolete entries, adding new entries, and adjusting shifted ranks. Since the affected set is approximate, the maintained index may differ slightly from a batch reconstruction; Section 5 shows that this discrepancy is negligible in practice. Periodic reconstruction can restore the batch-built state when needed.

Algorithm. Algorithm 5 incrementally maintains the HRNN index upon the arrival of a new point o_{new} , updating its three coupled structures: the HNSW graph G_{HNSW} , ranked KNN graph G_{KNN} , and reverse-neighbor lists R . In Phase 1, the algorithm inserts o_{new} into G_{HNSW} and reuses the search result $W(o_{\text{new}})$ obtained during insertion (Lines 1–2). The top- m_u closest points in $W(o_{\text{new}})$ are selected as proxies $N_{m_u}(o_{\text{new}})$ for identifying affected points. In Phase 2, the algorithm constructs the approximate affected set $\hat{S}_u(o_{\text{new}})$ (Lines 3–7). For each proxy b , it scans $R[b]$ in ascending rank order and collects owner points with ranks at most Θ_u . This proxy-based strategy avoids the costly computation of the exact affected set. In Phase 3, the algorithm initializes $G_{\text{KNN}}[o_{\text{new}}]$ with the top- K points in $W(o_{\text{new}})$ and inserts the corresponding entries into R (Lines 8–12). Finally, Phase 4 updates the neighborhoods of all points in $\hat{S}_u(o_{\text{new}})$ (Lines 13–23). For each affected point x , if $\delta(x, o_{\text{new}}) < \hat{r}_K(x)$, the algorithm inserts o_{new} into $G_{\text{KNN}}[x]$ and evicts the farthest neighbor if necessary. It then synchronizes R by removing obsolete entries and inserting or updating entries for the modified neighborhood, keeping G_{KNN} and R consistent after insertion.

Algorithm 5: HRNN Index Insertion Maintenance

Input: Index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, R)$, new vector o_{new} , update parameters m_u, Θ_u

Output: Updated index $\mathcal{I}$

// Phase 1: Insert into HNSW and collect proxy vectors

```

1  ( $G_{\text{HNSW}}, W[o_{\text{new}}]$ )  $\leftarrow$  HNSW-INSERT( $G_{\text{HNSW}}, o_{\text{new}}$ );
2   $N_{m_u}(o_{\text{new}}) \leftarrow$  top- $m_u$  closest vectors in  $W[o_{\text{new}}]$  to  $o_{\text{new}}$ ;
   // Phase 2: Construct the approximate affected set
3   $\hat{S}_u(o_{\text{new}}) \leftarrow \emptyset$ ;
4  for each  $b \in N_{m_u}(o_{\text{new}})$  do
5      for each  $(x, j) \in R[b]$  in ascending order of  $j$  do
6          if  $j > \Theta_u$  then break;
7           $\hat{S}_u(o_{\text{new}}) \leftarrow \hat{S}_u(o_{\text{new}}) \cup \{x\}$ ;

   // Phase 3: Initialize the neighbor list of new vector
8   $L_{\text{new}} \leftarrow$  top- $K$  vectors in  $W(o_{\text{new}})$  ranked by  $\delta(o_{\text{new}}, \cdot)$ ;
9   $G_{\text{KNN}}[o_{\text{new}}] \leftarrow L_{\text{new}}$ ;
10 for each  $v \in L_{\text{new}}$  do
11      $j \leftarrow$  rank of  $v$  in  $L_{\text{new}}$ ;
12     Insert  $(o_{\text{new}}, j)$  into  $R[v]$ , keeping  $R[v]$  sorted by rank;

   // Phase 4: Refresh affected neighborhoods
13 for each  $x \in \hat{S}_u(o_{\text{new}})$  do
14      $L_{\text{old}} \leftarrow G_{\text{KNN}}[x]$ ;
15      $\hat{r}_K(x) \leftarrow \delta(x, G_{\text{KNN}}[x, K])$ ;
16     if  $\delta(x, o_{\text{new}}) < \hat{r}_K(x)$  then
17          $L_{\text{upd}} \leftarrow \text{TopK}(L_{\text{old}} \cup \{o_{\text{new}}\})$  ranked by  $\delta(x, \cdot)$ ;
18          $G_{\text{KNN}}[x] \leftarrow L_{\text{upd}}$ ;
19         for each  $v \in L_{\text{old}} \setminus L_{\text{upd}}$  do
20             Remove  $(x, \cdot)$  from  $R[v]$ ;
21         for each  $v \in L_{\text{upd}}$  do
22              $j \leftarrow$  rank of  $v$  in  $L_{\text{upd}}$ ;
23             Insert or update  $(x, j)$  in  $R[v]$ , keeping  $R[v]$  sorted by rank;

24 return  $\mathcal{I}$ ;
```

THEOREM 4.11. For an insertion of o_{new} , the update incurs

$$O\left(C_H^{\text{ins}}(o_{\text{new}}) + s_u + d|\hat{S}_u(o_{\text{new}})| + (a_u + 1)K|D| \log |D|\right)$$

maintenance time, where $C_H^{\text{ins}}(o_{\text{new}})$ includes the HNSW insertion and extraction of the required search results, s_u is the number of scanned reverse-neighbor entries, $\hat{S}_u(o_{\text{new}})$ is the approximate affected set, and a_u is the number of updated ranked KNN lists.

PROOF. Inserting o_{new} into HNSW and retrieving the required ranked search results costs $O(C_H^{\text{ins}}(o_{\text{new}}))$. Scanning the reverse-neighbor entries and deduplicating candidate affected points costs $O(s_u)$. Testing whether o_{new} enters each candidate's ranked KNN list requires one d -dimensional distance computation, totaling $O(d|\hat{S}_u(o_{\text{new}})|)$. Using stored neighbor distances, updating each affected ranked KNN list costs $O(K)$ and induces at most $O(K)$ reverse-entry changes, including insertions, deletions, and rank adjustments. Initializing the new point's list induces another $O(K)$ changes, yielding $O((a_u + 1)K)$ reverse-entry changes in total. Since each reverse-neighbor list contains at most $|D|$ entries, locating and modifying an entry costs $O(|D|)$, while re-sorting the

list by rank costs $O(|D| \log |D|)$. Therefore, even if the entire reverse list is re-sorted after every change, synchronization costs at most $O((a_u + 1)\mathbf{K}|D| \log |D|)$, which also subsumes the ranked-list update cost. $\square$

Remark. Building HRNN solely through incremental insertions is feasible but incurs per-update maintenance costs. Unlike batch construction in Algorithm 4, which performs one-time NNDescent refinement, incremental construction repeatedly discovers affected sets and synchronizes reverse-neighbor lists. Its total cost therefore depends on the update proxies, affected-area sizes, and number of modified reverse-neighbor lists. Our experiments show that it is slower than batch construction on static datasets.

Point Deletion. Deleting a data point requires a different maintenance strategy from insertion. For insertion, HRNN uses RkNN-style search to approximate the affected set of points, updates their ranked KNN lists, and synchronizes the corresponding reverse-neighbor lists. In contrast, when deleting a point o , the affected points can be identified directly from $\mathbf{R}[o]$, as they are precisely those whose stored KNN lists contain o . We can therefore remove o from these lists and update $\mathbf{R}$ accordingly. The main challenge, however, is that removing o may leave some KNN lists with fewer than $\mathbf{K}$ entries. Restoring complete KNN lists requires searching for replacement neighbors, which can be expensive.

Lazy update. To avoid complete KNN list reconstruction, we adopt a lazy update strategy. For a batch $B \subseteq D$ of deleted points, we first mark them as deleted in the HNSW graph while retaining their graph links. Thus, deleted points are excluded from search results but remain in the graph to preserve connectivity, following the standard lazy-deletion strategy for HNSW. Let $D^+ = \{x \in D \mid x \text{ is not marked as deleted}\}$ denote the surviving points in D that are not deleted. We then remove deleted points from the ranked KNN lists of affected points (i.e., $\mathbf{R}[o]$) and rebuild $\mathbf{R}$ from the resulting KNN graph. We describe batch deletion for generality; single-point deletion is a special case with $|B| = 1$. This lazy update maintains consistency between the KNN lists and reverse-neighbor lists without restoring every KNN list to exactly $\mathbf{K}$ neighbors.

Query processing. For an RkNN query q , processing proceeds as before, except that we first identify $U = \{x \in D^+ \mid |G_{\text{KNN}}[x]| < k\}$, i.e., surviving points with fewer than k neighbors in their KNN lists. Since their stored lists cannot provide the required k NN-radius, points in U are excluded from verification until their KNN lists are reconstructed. For every remaining point in D^+ , the stored KNN list contains at least k surviving neighbors in distance order, preserving the one-sided verification property of Proposition 1. Moreover, the cost of identifying U can be amortized across queries with the same k . Points in U can be repaired through KNN list reconstruction or temporarily excluded. More generally, we rebuild the complete index when recall, precision, or throughput on a validation workload falls below the application’s target. Section 5 (Exp-9) shows that HRNN maintains its recall and precision targets and its relative advantage over the baselines even after deleting up to 50% of D . Our current implementation processes insertions and deletions in separate maintenance phases; supporting interleaved updates is left for future work.

Algorithm 6: HRNN Index Deletion Maintenance

Input: Index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$, point set B to delete
Output: Updated index $\mathcal{I}$
 // Phase 1: Mark deleted points while preserving links
 1 **for each** $x \in B$ **do**
 2 Mark x as deleted in G_{HNSW} ;
 3 $D^+ \leftarrow \{x \in D \mid x \text{ is not marked deleted}\}$;
 // Phase 2: Filter ranked neighbor lists
 4 **for each** $x \in D^+$ **in parallel do**
 5 $L_x \leftarrow \emptyset$;
 6 **for each** $(v, \delta(x, v)) \in G_{\text{KNN}}[x]$ **in stored order do**
 7 **if** $v \in D^+$ **then** Append $(v, \delta(x, v))$ to L_x ;
 8 $G_{\text{KNN}}[x] \leftarrow L_x$;
 // Phase 3: Rebuild reverse-neighbor lists
 9 Initialize $\mathbf{R}[v]$ for each $v \in D^+$;
 10 **for each** $x \in D^+$ **do**
 11 **for** $j \leftarrow 1$ **to** $|G_{\text{KNN}}[x]|$ **do**
 12 $v \leftarrow G_{\text{KNN}}[x, j]$;
 13 Append (x, j) to $\mathbf{R}[v]$;
 14 Sort each $\mathbf{R}[v]$ by rank j ;
 15 **return** $\mathcal{I}$;

Algorithm. Algorithm 6 maintains the HRNN index after a batch of deletions in three phases. First, it marks each point in B as deleted in G_{HNSW} while preserving its graph links for routing (Lines 1–2), and identifies the surviving point set D^+ (Line 3). Second, it filters the ranked KNN lists of surviving points in parallel (Line 4). For each point x , it initializes an empty list L_x (Line 5), scans the stored neighbors in their original order, and appends only surviving neighbors to L_x (Lines 6–7). It then replaces $G_{\text{KNN}}[x]$ with the filtered list, preserving the distance order without computing new distances (Line 8). Third, it initializes an empty reverse-neighbor list for each surviving point (Line 9) and traverses the filtered KNN lists (Lines 10–12). Whenever v is the j -th stored neighbor of x , it appends (x, j) to $\mathbf{R}[v]$ (Line 13). Finally, it sorts each reverse-neighbor list by the updated rank j (Line 14) and returns the updated index (Line 15).

THEOREM 4.12. *Deleting a point o incurs $O(|D|\mathbf{K} \log |D|)$ time.*

PROOF. Marking deleted points and identifying the surviving set D^+ takes $O(|D|)$ time. Filtering the ranked KNN lists examines at most $\mathbf{K}$ entries per surviving point, costing $O(|D|\mathbf{K})$. Rebuilding the reverse-neighbor lists processes each surviving neighbor entry once and thus also costs $O(|D|\mathbf{K})$. Let $r_v = |\mathbf{R}[v]|$. Sorting all reverse-neighbor lists costs $O(\sum_{v \in D^+} r_v \log r_v)$. Since $r_v \leq |D|$ and $\sum_{v \in D^+} r_v \leq |D|\mathbf{K}$, this is bounded by $O(|D|\mathbf{K} \log |D|)$. Hence, the overall time complexity is $O(|D|\mathbf{K} \log |D|)$. $\square$

5 EXPERIMENT

5.1 Experimental Setup

Datasets. We conduct experiments on eight real-world datasets: SIFT [30], MSong [5], GIST [27], MSMARCO [43], BookCorpus [10], EVA02 [10], ConvNeXt [10], and arXiv [23]. SIFT, MSong, and GIST are standard RkNN benchmarks [50]. MSMARCO, BookCorpus, and arXiv contain embeddings from modern text-embedding models, while EVA02 and ConvNeXt contain ImageNet [11] embeddings

Table 3: Dataset statistics.

Dataset	Dimension	# Base	# Query	Domain
SIFT	128	1.0M	10,000	Images
MSong	420	1.0M	1,000	Audio
GIST	960	1.0M	1,000	Images
MSMARCO	1024	1.0M	1,000	Text
BookCorpus	1024	1.0M	1,000	Text
EVA02	1024	1.28M	1,000	Images
ConvNeXt	1536	1.28M	1,000	Images
arXiv	4096	2.73M	1,000	Text

generated by the corresponding models [14, 62]. Table 3 summarizes the dataset statistics. The main experiments use datasets or subsets with up to 2.73 million base points (see # Base), while Exp-10 in the main paper and Exp-11 in the technical report [66] further evaluate scalability on datasets with up to 100 million points.

Compared Methods. We compare HRNN against representative graph-based approximate RkNN methods, including SFT, RDT, and the state-of-the-art HAMG [50]. Following HAMG, we implement SFT and RDT on top of HNSW to provide strong high-dimensional baselines. We do not compare against spatial-partitioning [51, 52, 63] or tree-based RkNN methods [1, 65], as prior studies have shown them to be prohibitively expensive on million-scale high-dimensional datasets.

- **HAMG.** The state-of-the-art ARkNN method [50]. Since the original Java implementation is not publicly available and the paper does not fully specify all dataset-specific parameter settings, we re-implement HAMG in C++ based on the published algorithm description. We tune both construction and query parameters over the ranges reported in [50], as well as larger budgets, and report the best recall-latency trade-off achieved on our platform.
- **HNSW-SFT.** The SFT-style baseline used in HAMG [48, 50]. It retrieves an expanded set of nearest neighbors of the query using HNSW and then verifies each candidate for RkNN membership.
- **HNSW-RDT.** The RDT baseline [6, 50] implemented on top of HNSW, following the design adopted in HAMG. It incrementally explores unchecked nearest neighbors around the query and applies the pruning and termination rules of RDT.

Metrics. We evaluate both efficiency and accuracy. Query efficiency is measured by average queries per second (QPS), while accuracy is measured by Recall@ k and Precision@ k , averaged over all queries. For index construction, we report build time and peak resident memory. Serving index space includes raw datasets and method-specific data structures, excluding system allocator overhead. In the scalability experiments, index construction exceeding 10^4 seconds is terminated.

Implementation. We implement all methods in C++ and compile them using GCC 13.1.0. Experiments are conducted on a server with an Intel Xeon Platinum 8352V CPU @ 2.10GHz and 512 GB RAM running Ubuntu 20.04 LTS. Following common practice in vector search studies [16, 26, 42, 50, 60], all query experiments are executed using a single thread, while index construction uses 64 threads. For HRNN, the HNSW graph is constructed with $M = 16$ and $ef_c = 400$. For baselines, we tune parameters according to the ranges reported in their original papers and use $C = 8000$ and $d_m = 80$ for HAMG, and $ef_c = 400$ for HNSW-RDT and HNSW-SFT.

Table 4: Indexing cost: time (Sec) and peak resident memory (GB).

Dataset	HNSW		HRNN		HAMG	
	Sec	GB	Sec	GB	Sec	GB
SIFT	22	1.40	45	2.98	545	2.02
MSong	50	3.71	94	4.51	1,097	4.20
GIST	150	8.06	207	8.86	2,926	8.67
MSMARCO	136	8.57	186	9.37	4,508	9.35
BookCorpus	161	8.57	216	9.37	4,868	9.04
EVA02	98	10.97	206	12.00	4,954	11.60
ConvNeXt	79	16.22	180	17.25	3,016	16.45
arXiv	1,174	90.61	1,712	92.80	21,865	92.11

Parameter Settings. Unless otherwise specified, all experiments use approximate RkNN search with $k = 10$ and a single HRNN index per dataset with $K = 100$. In each experiment, we vary the number of proxies $m \in \{1, 3, 5, 10, 20, 50, 100, 200, 400\}$ and the rank threshold $\Theta \in \{10, 50, 100\}$, with each (m, Θ) pair defining a parameter configuration. For a target recall or precision τ , we report the highest throughput among configurations achieving Recall@ $k \geq \tau$ or Precision@ $k \geq \tau$, respectively.

5.2 Experimental Results

Exp-1: Query Performance Comparison. We first compare the accuracy-throughput trade-offs of all methods. Fig. 11 reports Recall@10 versus QPS (top) and Precision@10 versus QPS (bottom) for HRNN, HAMG, HNSW-SFT, and HNSW-RDT across all eight datasets. Overall, HRNN consistently outperforms the baselines, with particularly large throughput gains at high recall and precision. For example, on arXiv at Recall@10 ≥ 0.99 , HRNN achieves 191.0 QPS, outperforming HAMG, HNSW-SFT, and HNSW-RDT by 117.0 $\times$, 28.4 $\times$, and 51.9 $\times$, respectively. Similarly, at its highest-recall configuration, HRNN maintains Precision@10 above 0.96 on all datasets, whereas HNSW-RDT drops to 0.834 on ConvNeXt, demonstrating HRNN’s advantage in both recall and precision.

To jointly evaluate recall and precision, we further consider the per-query F_1 score, defined as $\frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$, with F_1 set to 0 when both precision and recall are 0. Fig. 12 reports the average F_1 @10 versus QPS over all queries. HRNN achieves the highest QPS at every F_1 @10 level attained by the baselines. For example, at F_1 @10 ≥ 0.95 on arXiv, HRNN achieves 267.6 QPS, compared with 13.9, 7.4, and 2.4 QPS for HNSW-SFT, HNSW-RDT, and HAMG, respectively.

Exp-2: Query Time Breakdown. HRNN follows a filter-and-verification framework: it retrieves proxy points, scans their reverse-neighbor lists to generate candidates, and verifies these candidates. Accordingly, we decompose query time into *proxy retrieval*, *reverse-neighbor-list scanning*, and *candidate verification*. We examine configurations achieving Recall@10 and Precision@10 targets of 0.95 and 0.99 to quantify the cost of higher accuracy. Fig. 13 shows that, on SIFT, MSong, EVA02, ConvNeXt, and arXiv, total query time is only 0.64–5.3 ms and is dominated by proxy retrieval, as small candidate sets suffice. In contrast, GIST, MSMARCO, and BookCorpus require broader candidate coverage, with verification accounting for 65.1%–91.4% of query time at Recall@10 ≥ 0.99 . On MSMARCO, this share increases from 15.9% at 0.95 to 65.1% at 0.99. Reverse-neighbor-list scanning remains below 13.7% across all datasets because HRNN scans only rank-truncated entries.

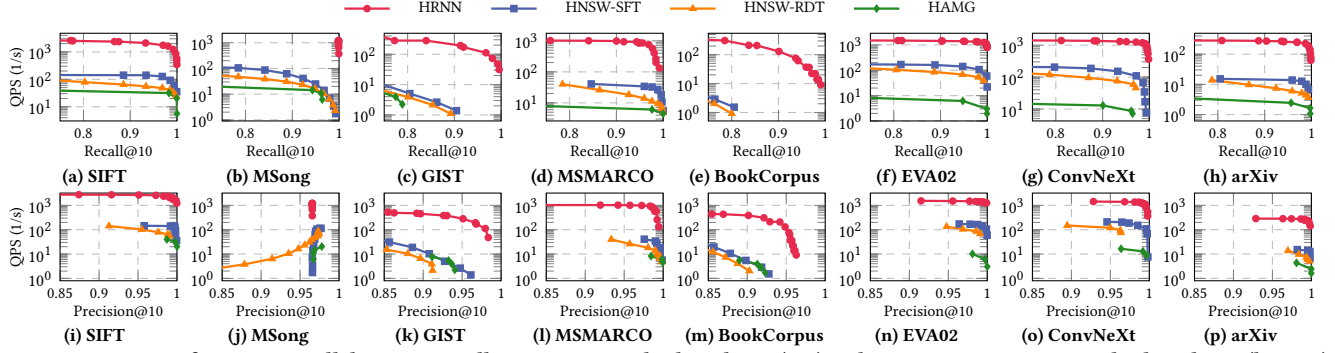


Figure 11: Query performance on all datasets: Recall@10 versus single-thread QPS (top) and Precision@10 versus single-thread QPS (bottom).

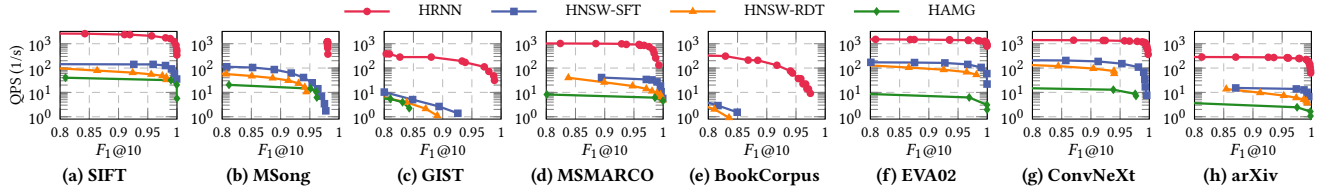


Figure 12: Query performance on all datasets: $F_1@10$ versus single-thread QPS.

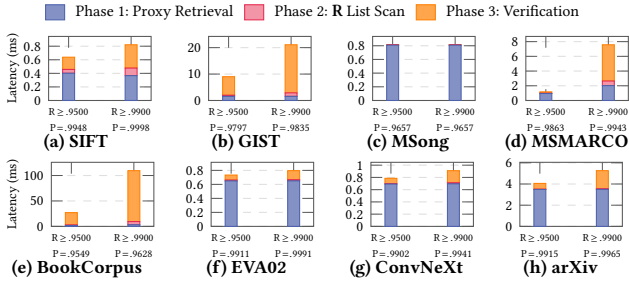


Figure 13: Query-time breakdown of HRNN. R and P below each bar indicate the target Recall@10 and Precision@10, respectively.

Table 5: Serving index space (MB). *Base* denotes raw vectors only.

Dataset	Base	HNSW	HAMG	HRNN	HRNN/Base
SIFT	512.0	616.7	722.5	1028.7	2.01×
MSong	1670.2	1757.5	1839.4	2167.1	1.30×
GIST	3840.0	3915.8	4043.9	4327.8	1.13×
MSMARCO	4096.0	4203.4	4375.3	4615.4	1.13×
BookCorpus	4096.0	4148.9	4253.9	4560.9	1.11×
EVA02	5247.7	5350.0	5463.2	5877.8	1.12×
ConvNeXt	7871.5	7954.9	7969.9	8482.8	1.08×
arXiv	44798.2	45063.4	45338.7	46189.9	1.03×

Table 6: k NN-radius and reverse-neighbor list ablation on GIST-1M.

Variant	Recall@10	Latency (ms)	Avg. Results
HRNN	0.9879	14.74	9.89
Exact Radius	0.9880	14.39	9.84
No Reverse-Neighbor Lists	0.9998	372.04	9.96

Exp-3: Index Construction Time and Space Consumption.

HRNN shifts part of query processing offline through index construction. We evaluate its construction time and peak resident memory in Table 4, and its serving space (including raw data and index structures) in Table 5. Table 4 compares the construction costs of HNSW, HRNN, and HAMG. Although HRNN incurs additional time cost over HNSW to construct the ranked KNN graph and reverse-neighbor lists, it is substantially cheaper to build than HAMG, which takes 11.7–24.2× longer across the eight datasets. Moreover, HRNN has peak resident memory comparable to the baselines during construction. Table 5 reports the serving space

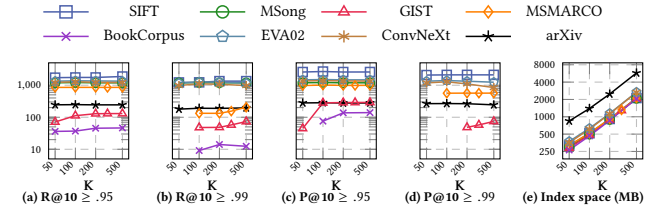


Figure 14: Sensitivity to parameter K . Panels (a)–(d) report the highest QPS satisfying the specified Recall@10 (R) or Precision@10 (P) target; panel (e) reports index size in MB.

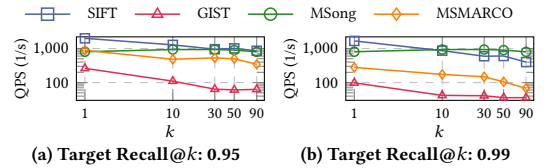


Figure 15: Performance of HRNN across different k values.

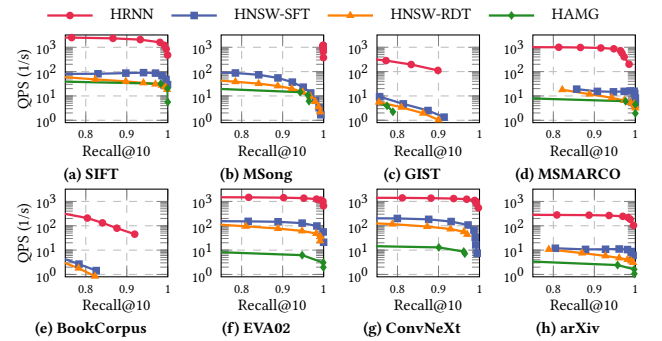


Figure 16: Equal-memory query performance in Recall@10.

during query processing, where *Base* denotes raw dataset storage. The relative overhead of HRNN is largest on low-dimensional SIFT and decreases for higher-dimensional datasets, from 2.01× the raw data size on SIFT to 1.03× on arXiv.

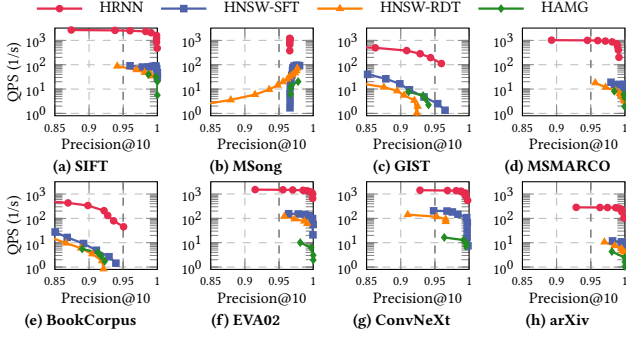


Figure 17: Equal-memory query performance in Precision@10.

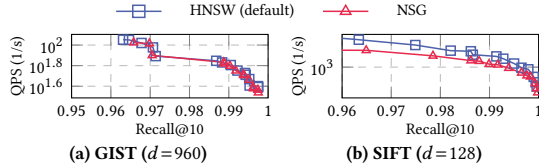


Figure 18: Effect of navigation graph choice (HNSW vs. NSG).

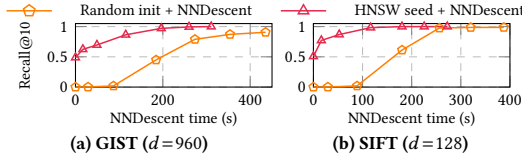


Figure 19: Effect of HNSW-seeding on KNN graph construction.

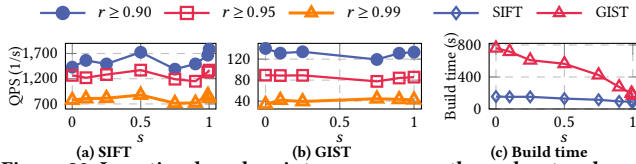


Figure 20: Insertion-based maintenance: query throughput and construction time. Panels (a)–(b) show throughput at Recall@10 targets of 0.90, 0.95, and 0.99; panel (c) shows total construction time.

Exp-4: Sensitivity to Parameter K . During index construction, the number of stored neighbors per point, K , is fixed and determines the supported query range $k \leq K$. To study its impact, we build each dataset with $K \in \{50, 100, 200, 500\}$. Fig. 14 reports the highest QPS satisfying each Recall or Precision target for each K . The minimum K required to meet a target varies by dataset. With $K = 50$, no (m, Θ) configuration achieves Recall@10 ≥ 0.99 on GIST, MSMARCO, or BookCorpus. Increasing K generally maintains or improves QPS; for example, increasing K from 100 to 500 improves QPS at Recall@10 ≥ 0.99 by 54.7% on MSMARCO. However, it also increases auxiliary index space (excluding raw vectors) by 4.08–4.44 $\times$ and construction time by 1.6–2.6 $\times$ across datasets. Overall, $K = 100$ provides a good cost–performance trade-off for our main experiments: it achieves high recall on datasets where $K = 50$ falls short while requiring substantially less auxiliary storage than $K = 500$. We thus set $K = 100$ by default.

Exp-5: Varying k . We evaluate whether a single HRNN index maintains high recall as k varies over $\{1, 10, 30, 50, 90\}$. For each dataset and k , we use the highest-throughput (m, Θ) configuration achieving Recall@ $k \geq 0.95$ and 0.99, respectively. Fig. 15 shows

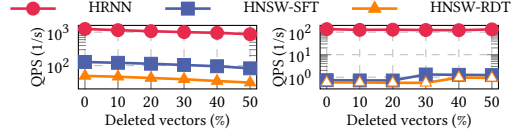


Figure 21: Deletion-based maintenance. Filled markers meet the panel-specific Precision@10 threshold; open markers do not.

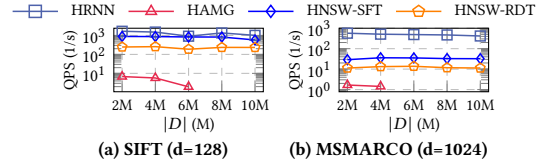


Figure 22: Cross-method scalability at Recall@10 ≥ 0.95 .

that HRNN achieves both recall targets for all tested k , although throughput often decreases as k increases. These results confirm that a single HRNN index supports varying k without rebuilding.

Exp-6: Equal-Memory Comparison. We evaluate whether HRNN retains its throughput advantage under matched query-stage resident memory. To equalize memory usage, we truncate HRNN’s reverse-neighbor lists and build denser HNSW graphs for SFT and RDT. Specifically, we retain entries with ranks up to Θ in reverse-neighbor lists of HRNN, using $\Theta = 50$ and $M = 43$ for SIFT and MSMARCO, and $\Theta = 10$ and $M = 23$ for the other six datasets. Figs. 16 and 17 show Recall@10 and Precision@10 versus QPS on all eight datasets. Table 7 reports per-dataset resident memory. Under matched memory budgets, HRNN retains higher throughput at comparable recall, showing that its advantage does not arise from higher memory usage. For example, on GIST, HRNN achieves 111.9 QPS at Recall@10 = 0.899, compared with 1.04 QPS for HNSW-RDT at Recall@10 = 0.900, demonstrating substantially higher efficiency of our HRNN at comparable recall.

Exp-7: Ablation Study. We examine three key design choices in our proposed HRNN index: (1) whether proxy retrieval depends on HNSW, (2) whether HNSW construction can provide effective seeds for G_{KNN} , and (3) whether materialized radius and reverse-neighbor lists enable efficient and accurate verification.

Navigation graph replacement. HRNN uses the navigation graph HNSW to locate proxies. We replace HNSW with NSG for proxy retrieval while keeping all other components unchanged. The results are in Fig. 18, and from the figure, we find that, on GIST, both achieve Recall@10 ≈ 0.99 at about 63 QPS, whereas on SIFT, HNSW is faster (1441.8 versus 1082.5 QPS at Recall@10 ≈ 0.99). Thus, HRNN is agnostic to the navigation graph, although its choice affects query efficiency.

KNN graph construction with HNSW seeding. Building HNSW already visits nearby candidates for each inserted point. We therefore reuse these candidates to initialize G_{KNN} and compare against the random initialization of standard NNDescent [13, 35]. Fig. 19 compares random and HNSW-seeded NNDescent. On GIST, random initialization reaches only 0.9066 after 435 seconds, whereas HNSW seeding reaches 1.0 in 310.85 seconds; SIFT shows a similar trend. Thus, HNSW candidates provide high-quality seeds that reduce refinement cost and improve G_{KNN} quality.

kNN-radius and reverse-neighbor list materialization. Reverse-neighbor lists determine the candidates to verify, while materialized radii enable efficient verification. We evaluate both roles by removing the reverse-neighbor lists and by replacing the estimated radii with exact ones in Table 6. Removing the reverse-neighbor lists requires verifying all 1M points, increasing latency from 14.74 ms to 372.04 ms. Replacing the G_{KNN} -derived radius with the exact brute-force radius (*Exact Radius*) changes recall only marginally, from 0.9879 to 0.9880, confirming the effectiveness of our design.

Exp-8: Insertion-Based Maintenance. As discussed in Section 4.4, HRNN supports both batch construction (Algorithm 4) and incremental insertion as new points arrive. We evaluate how these two strategies affect query performance. We vary the initial batch fraction s : the first $s|D|$ points are batch-built and the remainder are inserted incrementally, with $s = 1$ denoting pure batch construction and $s = 0$ pure insertion. For each s , we report the highest-QPS configuration satisfying each Recall@10 target. Fig. 20 shows that incremental insertion largely preserves the recall-QPS trade-off. On SIFT, pure insertion achieves 775 QPS at Recall@10 ≥ 0.99 , compared with 813 QPS for pure batch construction; GIST shows a similar trend at the 0.90 and 0.95 recall targets. The main overhead lies in construction, as each insertion synchronizes multiple index structures. Pure insertion takes $1.80\times$ and $4.47\times$ the batch construction time on SIFT and GIST, respectively, due to incremental updates to HNSW, G_{KNN} , and reverse-neighbor lists. However, even with $s = 0$ (pure insertion), our method maintains efficient index construction and query processing, demonstrating the effectiveness of the proposed index construction method.

Exp-9: Deletion-Based Maintenance. We evaluate HRNN’s deletion maintenance strategy, which uses lazy updates to reduce deletion cost but may leave some KNN lists with fewer than K neighbors. We therefore examine how much deletion the maintained index can tolerate before reconstruction becomes necessary. We evaluate the deletion algorithm in Section 4.4 on SIFT and GIST with 10–50% of points deleted uniformly at random. We exclude HAMG because adapting its statistical pruning strategy to deletions is non-trivial. Fig. 21 reports the fastest configuration achieving Recall@10 ≥ 0.95 on SIFT and Recall@10 ≥ 0.90 on GIST. Across deletion ratios up to 50%, HRNN remains $9.8\text{--}10.5\times$ faster than HNSW-SFT and $25.6\text{--}28.1\times$ faster than HNSW-RDT; moreover, HNSW-RDT fails to meet the Precision@10 target on GIST (open markers). Thus, HRNN can defer reconstruction while retaining its throughput advantage under substantial deletions.

Exp-10: Query Scalability. Our main experiments evaluate the query efficiency-accuracy trade-off on (subsets of) datasets with up to 2.73M vectors. We further evaluate the scalability of HRNN as the dataset grows to 10M vectors. We use SIFT and MSMARCO with $|D| \in \{2, 4, 6, 8, 10\}M$ and compare all methods at a common recall target using indexes with $K = 500$. We evaluate HRNN, HNSW-SFT, and HNSW-RDT at all dataset sizes. For HAMG, we report only completed runs, as construction exceeds 10^4 seconds on larger instances while query throughput remains below 10 QPS. Fig. 22 compares query throughput at Recall@10 ≥ 0.95 . On SIFT, HRNN is $1.10\text{--}1.88\times$ faster than HNSW-SFT and $4.35\text{--}6.76\times$ faster than

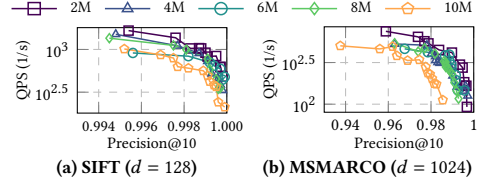


Figure 23: Precision@10 against single-thread QPS for HRNN across dataset sizes.

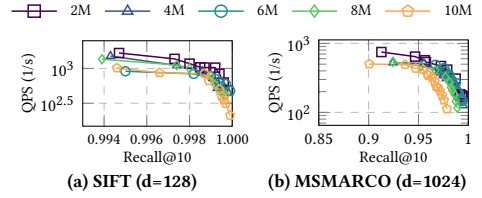


Figure 24: Scalability of HRNN across dataset sizes.

Table 7: Query-stage resource consumption (GB) under equal-memory settings.

Dataset	HRNN	HNSW-SFT	HNSW-RDT	HAMG
SIFT	0.978	0.981	0.981	0.990
MSong	1.970	1.974	1.974	2.073
GIST	4.144	4.148	4.148	4.229
MSMARCO	4.560	4.563	4.563	4.720
BookCorpus	4.400	4.404	4.404	4.400
ImageNet-EVA02	5.631	5.636	5.636	5.764
ImageNet-ConvNeXt	8.256	8.262	8.262	8.170
arXiv	45.614	45.625	45.625	46.071

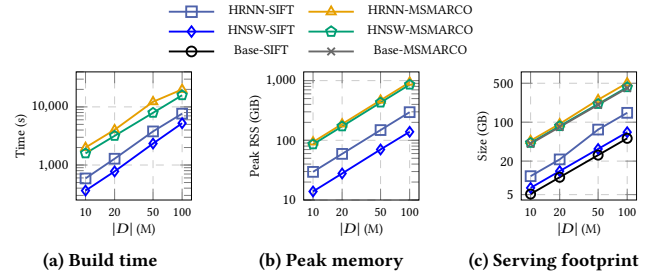


Figure 25: Scalability evaluation on SIFT and MSMARCO (100M).

HNSW-RDT across all dataset sizes, with an even larger advantage over HAMG. Thus, HRNN retains its throughput advantage as the dataset scales to 10M vectors. To evaluate query scalability, we vary the target recall and precision and measure QPS across different dataset scales. Figures 24 and 23 report the recall-QPS and Precision@10-QPS trade-offs, respectively. As shown in Fig. 24, HRNN maintains a stable recall-QPS trade-off as the dataset scales to 10M vectors. Similarly, Fig. 23 shows that both SIFT and MSMARCO retain high Precision@10 as the dataset grows, with only a slight downward shift in precision.

Exp-11: Construction Scalability. While Exp-10 evaluates query performance on datasets up to 10M vectors, we further examine construction scalability on SIFT and MSMARCO with $|D| \in \{10, 20, 50, 100\}M$. In particular, we investigate whether the ranked KNN graph and reverse-neighbor lists make HRNN prohibitively expensive to construct and store at larger scales. We measure construction time, peak memory, and serving index space up to 100M

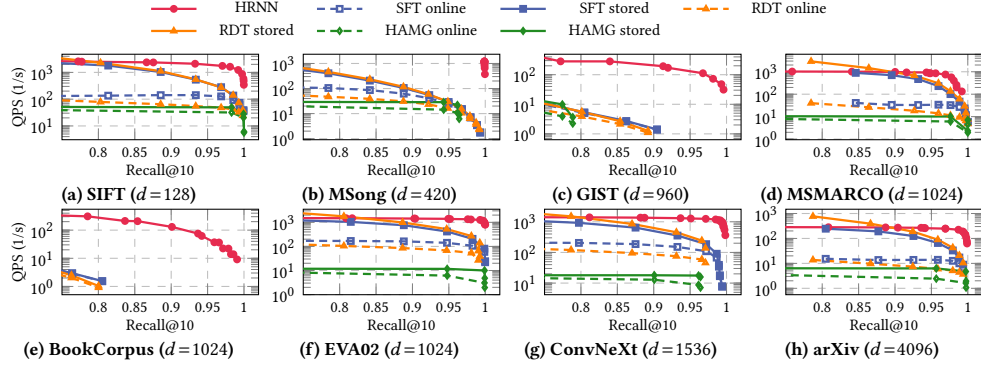


Figure 26: Recall@10-QPS comparison of baselines using online k NN-radius computation (dashed, open markers) or cached float32 $k = 10$ radius (solid, filled markers), against HRNN.

vectors, using HNSW as the reference for quantifying the additional overhead. We build HRNN and HNSW with the same HNSW parameters, with HRNN using $K = 100$ as in **Exp-4**. All experiments run on an Intel Xeon Platinum 8370C server with 1.5 TB RAM.

Fig. 25 reports the results. At 100M vectors, HRNN takes 2.13 hours and 296.1 GB peak memory on SIFT, and 5.54 hours and 936.1 GB on MSMARCO. Relative to HNSW, the build-time ratios are $1.46\times$ and $1.26\times$, the peak-memory ratios are $2.13\times$ and $1.09\times$, and the serving-footprint ratios are $2.23\times$ and $1.19\times$, respectively. From 10M to 100M vectors, peak memory grows approximately tenfold on both datasets, while build time increases by $13.0\times$ and $10.0\times$. Thus, HRNN remains feasible to construct on a single server at 100M scale, incurring only 26–46% additional build time over HNSW, with lower relative memory overhead on the higher-dimensional MSMARCO dataset.

Exp-12: Effect of Cached Radius. We further examine whether the baselines can match HRNN’s query efficiency when k NN-radius computation during verification is eliminated. For each baseline, we replace online k NN-radius computation with cached radius while keeping the index and query parameters unchanged. Fig. 26 shows that caching accelerates all baselines, but the gain diminishes as more candidates are expanded to achieve higher recall. At Recall@10 ≥ 0.95 , HNSW-RDT achieves a $5.7\text{--}21.9\times$ speedup, which drops to $2.2\text{--}3.6\times$ at Recall@10 ≥ 0.99 ; HNSW-SFT gains at most $1.4\times$. Even with cached radius, the best baseline achieves only 306.7 QPS at Recall@10 ≥ 0.95 on MSMARCO, compared with 855.8 QPS for HRNN. Thus, HRNN’s advantage also stems from candidate generation, where reverse-neighbor retrieval avoids the baselines’ neighborhood expansion.

Exp-13: Search Parameter Sensitivity. HRNN has two query-time parameters: the number of proxies m and the rank threshold Θ for scanning reverse-neighbor lists. We evaluate their effects on recall, precision, and throughput and identify the highest-throughput configurations at different accuracy targets. Fig. 27 reports the full (m, Θ) grid. Across all datasets, increasing either parameter generally improves Recall@10 and Precision@10 but reduces throughput, reflecting the trade-off between candidate coverage and query efficiency. The best configuration depends on the dataset and accuracy target, suggesting that m and Θ should be tuned jointly. Both can be

Table 8: Best HRNN parameter configurations for target recall@10.

Dataset	Recall@10 ≥ 0.95		Recall@10 ≥ 0.99	
	(m, Θ)	Recall / QPS	(m, Θ)	Recall / QPS
GIST	(50, 100)	0.9676 / 112.0	(200, 100)	0.9930 / 47.8
SIFT	(20, 100)	0.9703 / 1747.6	(50, 100)	0.9921 / 1228.3
MSMARCO	(100, 10)	0.9558 / 855.8	(400, 100)	0.9918 / 132.9
MSong	(1, 10)	0.9988 / 1213.9	(1, 10)	0.9988 / 1213.9
BookCorpus	(100, 50)	0.9593 / 37.0	(400, 100)	0.9905 / 9.1
EVA02	(20, 10)	0.9517 / 1373.7	(10, 50)	0.9929 / 1263.7
ConvNeXt	(1, 100)	0.9509 / 1279.2	(10, 50)	0.9917 / 1104.3
arXiv	(10, 50)	0.9691 / 247.5	(20, 100)	0.9929 / 191.0

adjusted without rebuilding the index. Table 8 reports the highest-throughput configurations achieving Recall@10 thresholds of 0.95 and 0.99. The optimal settings vary widely: MSong achieves Recall@10 ≥ 0.99 with $(m, \Theta) = (1, 10)$, whereas GIST, MSMARCO, and BookCorpus require (200, 100) or (400, 100). However, as described in Exp-1, we can select the configuration that meets a target accuracy, enabling HRNN to adapt effectively to different datasets.

6 RELATED WORK

As discussed in Section 3, existing Rk NN methods generally follow a filter-and-verification framework. They typically verify each candidate via a standard k NN search and mainly differ in candidate generation. Accordingly, we categorize them into three groups.

Classical Methods. Early studies focus on exact Rk NN search in low-dimensional spaces. Partition-based methods [51, 52, 63, 71] partition the space into geometric regions (e.g., half-spaces or influence zones) and prune candidates based on their positions relative to the query. Precomputation-based methods [1, 65] typically use spatial indexes (e.g., M-trees) with conservative subtree distance bounds for pruning. However, both suffer from the curse of dimensionality: spatial partitions lose discriminative power and approach large-scale scans, while precomputed bounds incur high index construction and maintenance costs.

Expansion-Based Methods. To scale Rk NN queries to high-dimensional vectors, expansion-based methods such as SFT [48] and RDT [6] were proposed. These methods aim to find approximate Rk NN results by progressively exploring nearby neighbors around the query vector q until a stopping condition is satisfied. As analyzed in Section 3, these methods become inefficient when nearby neighbors do not align well with the true Rk NN results, since they must explore a large neighborhood region to maintain high search accuracy.

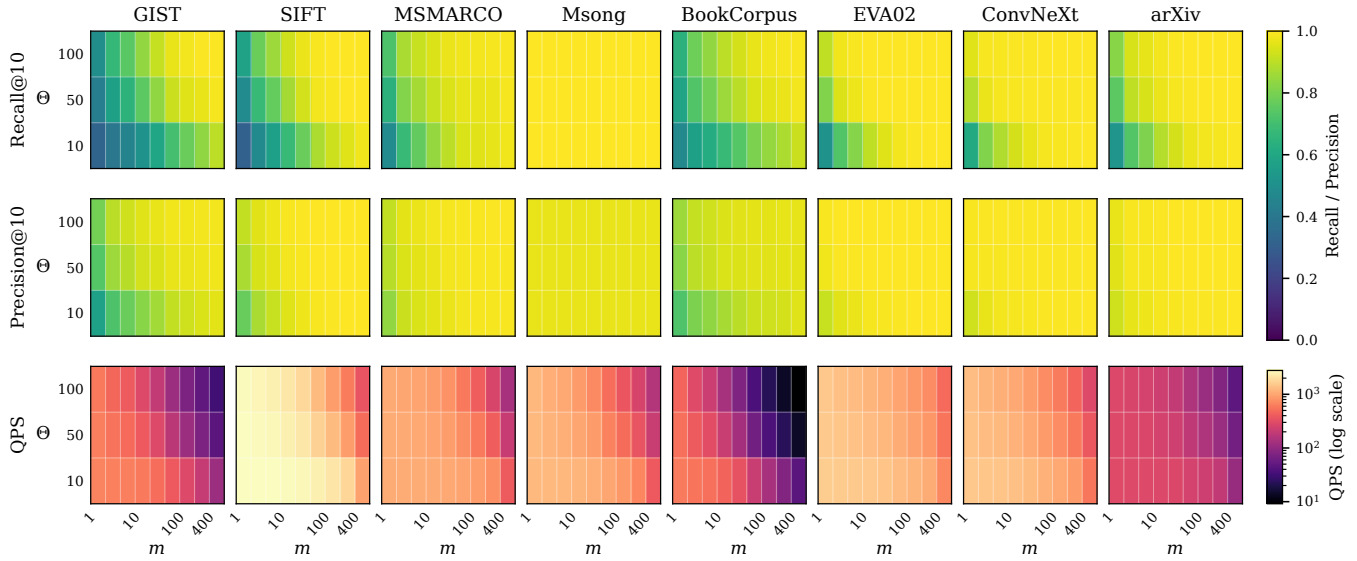


Figure 27: Recall@10, Precision@10, and QPS (log scale) across the evaluated (m, Θ) grid.

The State-of-the-Art and Beyond. Proximity graphs such as HNSW [42] and NSG [16] dominate high-dimensional approximate k NN search. The state-of-the-art HAMG [50] adapts HNSW to approximate the MRN graph, whose key property guarantees that true Rk NN results lie within the query’s k -hop neighborhood. HAMG exploits this property to retrieve graph neighbors as candidates. Although this heuristic weakens MRN’s theoretical guarantees, it remains effective in practice. However, the explored graph neighborhood can be much larger than the true Rk NN result set, yielding many unnecessary candidates. A large body of work has studied high-dimensional k NN search, including distance computation optimization [12, 17, 69, 70], novel index structures [18, 53, 59], and search optimizations [9, 19, 29, 34, 39]. As we focus on Rk NN search, we omit a detailed discussion of these methods.

7 CONCLUSION

This paper studies approximate Rk NN search over high-dimensional vectors. Motivated by the limitations of existing methods, we propose HRNN, a hybrid graph index that treats nearby neighbors as proxy points and materializes k NN-radius offline to avoid costly online computation. HRNN integrates a navigation graph, a ranked KNN graph, and reverse-neighbor lists, together with efficient algorithms for index construction and maintenance. Experiments on real-world datasets demonstrate that HRNN consistently outperforms existing methods in both efficiency and scalability while maintaining high search accuracy.

REFERENCES

- [1] Elke Achtert, Christian Böhm, Peer Kröger, Peter Kunath, Alexey Pryakhin, and Matthias Renz. 2006. Efficient reverse k -nearest neighbor search in arbitrary metric spaces. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data* (Chicago, IL, USA) (SIGMOD ’06). Association for Computing Machinery, New York, NY, USA, 515–526. <https://doi.org/10.1145/1142473.1142531>
- [2] Burouj Armgaan, Eshan Jain, Harsh Pandey, Mahesh Chandran, and Sayan Ranu. 2025. GnnXemplar: Exemplars to Explanations - Natural Language Rules for Global GNN Interpretability. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=eafjjoZAHm>
- [3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=hSyW5go0v8>
- [4] Ilias Azizi, Karima Echihiabi, and Themis Palpanas. 2023. Elpis: Graph-based similarity search for scalable data science. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1548–1559.
- [5] Thierry Bertin-Mahieux, Daniel P.W. Ellis, Brian Whitman, and Paul Lamere. 2011. The Million Song Dataset. In *Proceedings of the 12th International Conference on Music Information Retrieval (ISMIR 2011)*.
- [6] Guillaume Casanova, Elias Englmeier, Michael E. Houle, Peer Kröger, Michael Nett, Erich Schubert, and Arthur Zimek. 2017. Dimensional testing for reverse k -nearest neighbor search. *Proc. VLDB Endow.* 10, 7 (March 2017), 769–780. <https://doi.org/10.14778/3067421.3067426>
- [7] Muhammad Aamir Cheema, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2011. Influence zone: Efficiently processing reverse k nearest neighbors queries. In *Proceedings of the 2011 IEEE 27th International Conference on Data Engineering (ICDE ’11)*. IEEE Computer Society, USA, 577–588. <https://doi.org/10.1109/ICDE.2011.5767904>
- [8] Meng Chen, Kai Zhang, Zhenying He, Yanan Jing, and X. Sean Wang. 2024. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 17, 11 (July 2024), 2735–2749. <https://doi.org/10.14778/3681954.3681959>
- [9] Patrick Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference 2023* (Austin, TX, USA) (WWW ’23). Association for Computing Machinery, New York, NY, USA, 3225–3235. <https://doi.org/10.1145/3543507.3583318>
- [10] Tingyang Chen, Cong Fu, Jiahua Wu, Haotian Wu, Hua Fan, Xiangyu Ke, Yunjun Gao, Yabo Ni, and Anxiang Zeng. 2026. Reveal Hidden Pitfalls and Navigate Next Generation of Vector Similarity Search from Task-Centric Views: [Experiments & Analysis]. *Proc. ACM Manag. Data* 4, 1, Article 76 (April 2026), 25 pages. <https://doi.org/10.1145/3786690>
- [11] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. IEEE, 248–255.
- [12] Liwei Deng, Penghao Chen, Ximu Zeng, Tianfu Wang, Yan Zhao, and Kai Zheng. 2024. Efficient Data-Aware Distance Comparison Operations for High-Dimensional Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 3 (Nov. 2024), 812–821. <https://doi.org/10.14778/3712221.3712244>
- [13] Wei Dong, Charikar Moses, and Kai Li. 2011. Efficient k -nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International Conference on World Wide Web* (Hyderabad, India) (WWW ’11). Association for Computing Machinery, New York, NY, USA, 577–586. <https://doi.org/10.1145/1963405.1963487>
- [14] Yuxin Fang, Quan Sun, Xinggang Wang, Tiejun Huang, Xinlong Wang, and Yue Cao. 2024. Eva-02: A visual representation for neon genesis. *Image and Vision Computing* 149 (2024), 105171.

- [15] Cong Fu, Changxu Wang, and Deng Cai. 2022. High Dimensional Similarity Search With Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 8 (2022), 4139–4150. <https://doi.org/10.1109/TPAMI.2021.3067706>
- [16] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.* 12, 5 (Jan. 2019), 461–474. <https://doi.org/10.14778/3303753.3303754>
- [17] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proc. ACM Manag. Data* 1, 2, Article 137 (June 2023), 27 pages. <https://doi.org/10.1145/3589282>
- [18] Yutong Gou, Jianyang Gao, Yuxuan Xu, and Cheng Long. 2025. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 1, Article 80 (Feb. 2025), 26 pages. <https://doi.org/10.1145/3709730>
- [19] Hao Guo and Youyou Lu. 2026. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. *ACM Trans. Storage* 22, 2, Article 12 (March 2026), 30 pages. <https://doi.org/10.1145/3793926>
- [20] Mridul Gupta, Samyak Jain, Vansh Ramani, Hariprasad Kodamana, and Sayan Ranu. 2025. Bonsai: Gradient-free graph condensation for node classification. In *International Conference on Learning Representations*, Vol. 2025. 66489–66509.
- [21] Kiana Hajebi, Yasin Abbasi-Yadkori, Hossein Shahbazi, and Hong Zhang. 2011. Fast approximate nearest-neighbor search with k-nearest neighbor graph. In *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence - Volume Volume Two* (Barcelona, Catalonia, Spain) (IJCAI'11). AAAI Press, 1312–1317.
- [22] Yaochen Huang, Zhihui Lai, Heng Kong, and Jie Xu. 2026. Reverse Neighborhood Discriminant Analysis for Feature Extraction. *Information Sciences* 756 (2026), 123831. <https://doi.org/10.1016/j.ins.2026.123831>
- [23] Patrick Iff, Paul Brügger, Marc Chrapek, David Kochergin, Maciej Besta, and Torsten Hoeft. 2026. Benchmarking Filtered Approximate Nearest Neighbor Search Algorithms on Transformer-based Embedding Vectors. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Australia) (SIGIR '26). Association for Computing Machinery, New York, NY, USA, 610–622. <https://doi.org/10.1145/3805712.3809731>
- [24] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 604–613.
- [25] Piotr Indyk and Haïke Xu. 2023. Worst-case performance of popular approximate nearest neighbor search implementations: guarantees and limitations. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 2891, 18 pages.
- [26] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf
- [27] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (Jan. 2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [28] Zhongming Jin, Debing Zhang, Yao Hu, Shiding Lin, Deng Cai, and Xiaofei He. 2014. Fast and accurate hashing via iterative nearest neighbors expansion. *IEEE transactions on cybernetics* 44, 11 (2014), 2167–2177.
- [29] Sungjun Jung, Yongsang Park, Haeun Lee, Young H. Oh, and Jae W. Lee. 2025. Angular Distance-Guided Neighbor Selection for Graph-Based Approximate Nearest Neighbor Search. In *Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) (WWW '25). Association for Computing Machinery, New York, NY, USA, 4014–4023. <https://doi.org/10.1145/3696410.3714870>
- [30] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in one billion vectors: Re-rank with source coding. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 861–864. <https://doi.org/10.1109/ICASSP.2011.5946540>
- [31] Noo-Ri Kim, Jin-Seop Lee, and Jee-Hyong Lee. 2024. Learning with Structural Labels for Learning with Noisy Labels. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 27600–27610. <https://doi.org/10.1109/CVPR52733.2024.02607>
- [32] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- [33] Binhong Li, Xiao Yan, and Shangqi Lu. 2026. Fast-Convergent Proximity Graphs for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 4, 1, Article 36 (April 2026), 24 pages. <https://doi.org/10.1145/3786650>
- [34] Liang Li, Shufeng Gong, Yanan Yang, Yiduo Wang, and Jie Wu. 2026. I/O Optimizations for Graph-Based Disk-Resident Approximate Nearest Neighbor Search: A Design Space Exploration. <https://doi.org/10.48550/arXiv.2602.21514> [cs]
- [35] Zhonggen Li, Xiangyu Ke, Yifan Zhu, Bocheng Yu, Baihua Zheng, and Yunjun Gao. 2025. Scalable Graph Indexing using GPUs for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 6, Article 360 (Dec. 2025), 27 pages. <https://doi.org/10.1145/3769825>
- [36] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *Proc. VLDB Endow.* 18, 4 (Dec. 2024), 1118–1130. <https://doi.org/10.14778/3717755.3717770>
- [37] Yuanbang Liang, Jing Wu, Yu-Kun Lai, and Yipeng Qin. 2022. Exploring and exploiting hubness priors for high-quality GAN latent sampling. In *International Conference on Machine Learning*. PMLR, 13271–13284.
- [38] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2021. HVS: hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search. *Proceedings of the VLDB Endowment* 15, 2 (2021), 246–258.
- [39] Kejing Lu, Chuan Xiao, and Yoshiharu Ishikawa. 2024. Probabilistic routing for graph-based approximate nearest neighbor search. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 1347, 19 pages.
- [40] Shangqi Lu and Yufei Tao. 2025. Proximity Graphs for Similarity Search: Fast Construction, Lower Bounds, and Euclidean Separation. *Proc. ACM Manag. Data* 3, 5, Article 280 (Nov. 2025), 25 pages. <https://doi.org/10.1145/3767716>
- [41] Tengfei Ma, Weiran Pan, and Wei Wei. 2026. TANGO: Text-Anchored Guided Optimization for Robust Fine-tuning Vision-Language Models under Label Noise. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 39186–39196.
- [42] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (April 2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [43] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A Human Generated Machine Reading Comprehension Dataset. In *CoCo@NIPS (CEUR Workshop Proceedings)*, Tarek Richard Besold, Antoine Bordes, Artur S. d'Ávila Garcez, and Greg Wayne (Eds.), Vol. 1773. CEUR-WS.org. <http://dblp.uni-trier.de/db/conf/nips/coco2016.html#NguyenRSGTMD16>
- [44] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 3, Article 120 (May 2024), 27 pages. <https://doi.org/10.1145/3654923>
- [45] Fengchao Peng, Chao Wang, Jianzhuang Liu, and Zhen Yang. 2021. Active Learning for Lane Detection: A Knowledge Distillation Approach. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. 15132–15141. <https://doi.org/10.1109/ICCV48922.2021.01487>
- [46] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases. *Proc. ACM Manag. Data* 1, 1, Article 54 (May 2023), 27 pages. <https://doi.org/10.1145/3588908>
- [47] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. 2010. Hubs in Space: Popular Nearest Neighbors in High-Dimensional Data. *J. Mach. Learn. Res.* 11 (Dec. 2010), 2487–2531.
- [48] Amit Singh, Hakan Ferhatosmanoglu, and Ali Şaman Tosun. 2003. High dimensional reverse nearest neighbor queries. In *Proceedings of the Twelfth International Conference on Information and Knowledge Management* (New Orleans, LA, USA) (CIKM '03). Association for Computing Machinery, New York, NY, USA, 91–98. <https://doi.org/10.1145/956863.956882>
- [49] Anbang Song, Ziqiang Yu, Wei Liu, Yating Xu, and Mingjin Tao. 2025. BRkNN-light: Batch Processing of Reverse k-Nearest Neighbor Queries for Moving Objects on Road Networks. In *Proceedings of the 19th International Symposium on Spatial and Temporal Data (SSTD '25)*. Association for Computing Machinery, New York, NY, USA, 80–89. <https://doi.org/10.1145/3748777.3748791>
- [50] Yitong Song, Kai Wang, Bin Yao, Zhida Chen, Jiong Xie, and Feifei Li. 2024. Efficient Reverse k Approximate Nearest Neighbor Search Over High-Dimensional Vectors. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 4262–4274. <https://doi.org/10.1109/ICDE60146.2024.00325>
- [51] Yufei Tao, Dimitris Papadias, and Xiang Lian. 2004. Reverse kNN search in arbitrary dimensionality. In *Proceedings of the Thirtieth International Conference on Very Large Data Bases - Volume 30* (Toronto, Canada) (VLDB '04). VLDB Endowment, 744–755.
- [52] Yufei Tao, Dimitris Papadias, Xiang Lian, and Xiaokui Xiao. 2007. Multidimensional reverse kNN search. *The VLDB Journal* 16, 3 (July 2007), 293–316. <https://doi.org/10.1007/s00778-005-0168-2>
- [53] Kento Tatsuno, Daisuke Miyashita, Taiga Ikeda, Kiyoshi Ishiyama, Kazunari Sumiyoshi, and Jun Deguchi. 2025. AiSAQ: All-in-Storage ANNS with Product Quantization for DRAM-free Information Retrieval. [arXiv:2404.06004](https://arxiv.org/abs/2404.06004) [cs.IR] <https://arxiv.org/abs/2404.06004>

- [54] Nenad Tomasev, Milos Radovanovic, Dunja Mladenic, and Mirjana Ivanovic. 2014. The Role of Hubness in Clustering High-Dimensional Data. *IEEE Transactions on Knowledge and Data Engineering* 26, 3 (2014), 739–751. <https://doi.org/10.1109/TKDE.2013.25>
- [55] Hongya Wang, Wenlong Wu, Cong Luo, Aobei Bian, Chunguang Meng, Yishuo Wu, and Ji Sun. 2025. Boosting Accuracy and Efficiency for Vector Retrieval with Local Scaling Graph. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 336–348. <https://doi.org/10.1109/ICDE65448.2025.00032>
- [56] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, and Lu Chen. 2024. An Interactive Multi-Modal Query Answering System with Retrieval-Augmented Large Language Models. *Proc. VLDB Endow.* 17, 12 (Aug. 2024), 4333–4336. <https://doi.org/10.14778/3685800.3685868>
- [57] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *Proc. ACM Manag. Data* 3, 3, Article 123 (June 2025), 29 pages. <https://doi.org/10.1145/3725260>
- [58] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *Proc. ACM Manag. Data* 3, 3, Article 123 (June 2025), 29 pages. <https://doi.org/10.1145/3725260>
- [59] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proc. ACM Manag. Data* 2, 1, Article 14 (March 2024), 27 pages. <https://doi.org/10.1145/3639269>
- [60] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (July 2021), 1964–1978. <https://doi.org/10.14778/3476249.3476255>
- [61] Zeyu Wang, Peng Wang, Themis Palpanas, and Wei Wang. 2023. Graph- and Tree-based Indexes for High-dimensional Vector Similarity Search: Analyses, Comparisons, and Future Directions. *IEEE Data Eng. Bull.* 47, 3 (2023), 3–21. <http://sites.computer.org/debull/A23sept/p3.pdf>
- [62] Sanghyun Woo, Shoubhik Debnath, Ronghang Hu, Xinlei Chen, Zhuang Liu, In So Kweon, and Saining Xie. 2023. Convnext v2: Co-designing and scaling convnets with masked autoencoders. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Ieee, 16133–16142.
- [63] Wei Wu, Fei Yang, Chee-Yong Chan, and Kian-Lee Tan. 2008. Finch: Evaluating reverse k-nearest-neighbor queries on location data. *Proceedings of the VLDB Endowment* 1, 1 (2008), 1056–1067.
- [64] Zheng Wu, Yitong Song, Xuliang Zhu, Huiling Li, Jianliang Xu, and Xin Huang. 2026. Balancing Global and Local: Representative Sampling for Large-Scale Vector Data. *Proc. ACM Manag. Data* 4, 3, Article 142 (May 2026), 27 pages. <https://doi.org/10.1145/3802019>
- [65] Chenyi Xia, Wynne Hsu, and Mong Li Lee. 2005. ERkNN: efficient reverse k-nearest neighbors retrieval with local kNN-distance estimation. In *Proceedings of the 14th ACM International Conference on Information and Knowledge Management (Bremen, Germany) (CIKM '05)*. Association for Computing Machinery, New York, NY, USA, 533–540. <https://doi.org/10.1145/1099554.1099697>
- [66] Wenxuan Xia, Mingyu Yang, Wentao Li, and Wei Wang. 2026. *Technical Report*. https://github.com/gegeji/hrnn/blob/main/technical_report.pdf
- [67] Jiadong Xie, Jeffrey Xu Yu, and Yingfan Liu. 2025. Graph based k-nearest neighbor search revisited. *ACM Transactions on Database Systems* 50, 4 (2025), 1–30.
- [68] Congyun Yang and King-Ip Lin. 2001. An index structure for efficient reverse nearest neighbor queries. In *Proceedings 17th International Conference on Data Engineering*. IEEE, 485–492.
- [69] Mingyu Yang, Liuchang Jing, Wentao Li, and Wei Wang. 2026. Quantization Meets Projection: A Happy Marriage for Approximate k-Nearest Neighbor Search. [arXiv:2411.06158 \[cs.DB\]](https://arxiv.org/abs/2411.06158) <https://arxiv.org/abs/2411.06158>
- [70] Mingyu Yang, Wentao Li, Jiabao Jin, Xiaoyao Zhong, Xiangyu Wang, Zhitao Shen, Wei Jia, and Wei Wang. 2025. Effective and General Distance Computation for Approximate Nearest Neighbor Search. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 1098–1110. <https://doi.org/10.1109/ICDE65448.2025.00087>
- [71] Shiyu Yang, Muhammad Aamir Cheema, Xuemin Lin, and Ying Zhang. 2014. SLICE: Reviving regions-based pruning for reverse k nearest neighbors queries. In *2014 IEEE 30th International Conference on Data Engineering*. 760–771. <https://doi.org/10.1109/ICDE.2014.6816698>
- [72] Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. 2024. Making Retrieval-Augmented Language Models Robust to Irrelevant Context. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=ZS4m74kZpH>
- [73] Xiao Yu, Hui Liu, Yan Zhang, Yuxiu Lin, and Caiming Zhang. 2024. Hubness-Enabled Clustering and Recovery for Large-Scale Incomplete Multi-View Data. *ACM Trans. Knowl. Discov. Data* 19, 1, Article 2 (Nov. 2024), 23 pages. <https://doi.org/10.1145/3694689>
- [74] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. 2024. RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=S1fc92uemC>
- [75] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proc. VLDB Endow.* 16, 8 (April 2023), 1979–1991. <https://doi.org/10.14778/3594512.3594527>
- [76] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, et al. 2025. VSAG: An Optimized Search Framework for Graph-Based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5017–5030.
- [77] Fa Zhu, Junbin Gao, Jian Yang, and Ning Ye. 2022. Neighborhood linear discriminant analysis. *Pattern Recogn.* 123, C (March 2022), 9. <https://doi.org/10.1016/j.patcog.2021.108422>